

1 基础算法

1.1 头文件

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  #define int long long
4  template<class... Args> void _(Args... args) {
5      auto _ = [&](auto x) { cout << x << " "; };
6      cout << "---->";
7      int arr[] = {_(args), 0}...};
8      cout << "\n";
9  }
10
11 signed main() {
12     int Task = 1;
13     for (cin >> Task; Task; Task--) {}
14 }
15 int __OI_INIT__ = []() {
16     ios::sync_with_stdio(0), cin.tie(0);
17     cout.tie(0);
18     cout << fixed << setprecision(12);
19     return 0;
20 }();

```

1.2 常用函数

```

1  i64 mysqrt(i64 n) { // 针对 sqrt 无法精确计算 ll 型
2      i64 ans = sqrt(n);
3      while ((ans + 1) * (ans + 1) <= n) ans++;
4      while (ans * ans > n) ans--;
5      return ans;
6  }
7  int mylcm(int x, int y) {
8      return x / gcd(x, y) * y;
9  }

```

```

1  template<class T> int log2floor(T n) { // 针对 log2 无法精确计算 ll 型；向下取整
2      assert(n > 0);
3      for (T i = 0, chk = 1;; i++, chk *= 2) {
4          if (chk <= n && n < chk * 2) {
5              return i;
6          }
7      }
8  }
9  template<class T> int log2ceil(T n) { // 向上取整
10     assert(n > 0);
11     for (T i = 0, chk = 1;; i++, chk *= 2) {
12         if (n <= chk) {
13             return i;
14         }
15     }
16 }
17 int log2floor(int x) {
18     return 31 - __builtin_clz(x);
19 }
20 int log2ceil(int x) { // 向上取整
21     return log2floor(x) + (__builtin_popcount(x) != 1);
22 }

```

```

1  template <class T> T sign(const T &a) {
2      return a == 0 ? 0 : (a < 0 ? -1 : 1);
3  }
4  template <class T> T floor(const T &a, const T &b) { // 注意大数据计算时会丢失精度
5      T A = abs(a), B = abs(b);
6      assert(B != 0);
7      return sign(a) * sign(b) > 0 ? A / B : -(A + B - 1) / B;
8  }
9  template <class T> T ceil(const T &a, const T &b) { // 注意大数据计算时会丢失精度
10     T A = abs(a), B = abs(b);
11     assert(b != 0);
12     return sign(a) * sign(b) > 0 ? (A + B - 1) / B : -A / B;
13 }

```

1.3 最大公约数 gcd

速度不如内置函数！ 以 $\mathcal{O}(\log(a+b))$ 的复杂度求解最大公约数。与内置函数 `__gcd` 功能基本相同（支持 $a, b \leq 0$ ）。有使用位运算的**常数优化版本**。

```

1  inline int mygcd(int a, int b) { return b ? gcd(b, a % b) : a; }

```

1.4 整数域二分

1.4.1 旧版（无法处理负数情况）

- 在递增序列 a 中查找 $\geq x$ 数中最小的一个（即 x 或 x 的后继）

```

1  while (l < r) {
2      int mid = (l + r) / 2;
3      if (a[mid] >= x) {
4          r = mid;
5      } else {
6          l = mid + 1;
7      }
8  }
9  return a[l];

```

- 在递增序列 a 中查找 $\leq x$ 数中最大的一个（即 x 或 x 的前驱）

```

1  while (l < r) {
2      int mid = (l + r + 1) / 2;
3      if (a[mid] <= x) {
4          l = mid;
5      } else {
6          r = mid - 1;
7      }
8  }
9  return a[l];

```

1.4.2 新版

- x 或 x 的后继

```

1  int l = 0, r = 1E8, ans = r;
2  while (l <= r) {
3      int mid = (l + r) / 2;
4      if (judge(mid)) {
5          r = mid - 1;
6          ans = mid;
7      } else {
8          l = mid + 1;
9      }
10 }
11 return ans;

```

- x 或 x 的前驱

```

1  int l = 0, r = 1E8, ans = 1;
2  while (l <= r) {
3      int mid = (l + r) / 2;
4      if (judge(mid)) {
5          l = mid + 1;
6          ans = mid;
7      } else {
8          r = mid - 1;
9      }
10 }
11 return ans;

```

1.5 实数域二分

目前主流的写法是限制二分次数。

```

1  for (int t = 1; t <= 100; t++) {
2      ld mid = (l + r) / 2;
3      if (judge(mid)) r = mid;
4      else l = mid;
5  }
6  cout << l << endl;

```

1.6 整数域三分

```

1  while (l < r) {
2      int mid = (l + r) / 2;
3      if (check(mid) <= check(mid + 1)) r = mid;
4      else l = mid + 1;
5  }
6  cout << check(l) << endl;

```

1.7 实数域三分

限制次数实现。

```
1  ld l = -1E9, r = 1E9;
2  for (int t = 1; t <= 100; t++) {
3      ld mid1 = (l * 2 + r) / 3;
4      ld mid2 = (l + r * 2) / 3;
5      if (judge(mid1) < judge(mid2)) {
6          r = mid2;
7      } else {
8          l = mid1;
9      }
10 }
11 cout << l << endl;
```

/END/

2 树上问题

2.1 树的直径

```

1 struct Tree {
2     int n;
3     vector<vector<int>> ver;
4     Tree(int n) {
5         this->n = n;
6         ver.resize(n + 1);
7     }
8     void add(int x, int y) {
9         ver[x].push_back(y);
10        ver[y].push_back(x);
11    }
12    int getlen(int root) { // 获取x所在树的直径
13        map<int, int> dep; // map用于优化输入为森林时的深度计算, 亦可用vector
14        function<void(int, int)> dfs = [&](int x, int fa) -> void {
15            for (auto y : ver[x]) {
16                if (y == fa) continue;
17                dep[y] = dep[x] + 1;
18                dfs(y, x);
19            }
20            if (dep[x] > dep[root]) {
21                root = x;
22            }
23        };
24        dfs(root, 0);
25        int st = root; // 记录直径端点
26
27        dep.clear();
28        dfs(root, 0);
29        int ed = root; // 记录直径另一端点
30
31        return dep[root];
32    }
33 };

```

2.2 树论大封装（直径+重心+中心）

```

1 struct Tree {
2     int n;
3     vector<vector<pair<int, int>>> e;
4     vector<int> dep, parent, maxdep, d1, d2, s1, s2, up;
5     Tree(int n) {
6         this->n = n;
7         e.resize(n + 1);
8         dep.resize(n + 1);
9         parent.resize(n + 1);
10        maxdep.resize(n + 1);
11        d1.resize(n + 1);
12        d2.resize(n + 1);
13        s1.resize(n + 1);
14        s2.resize(n + 1);
15        up.resize(n + 1);
16    }
17    void add(int u, int v, int w) {
18        e[u].push_back({w, v});
19        e[v].push_back({w, u});
20    }
21    void dfs(int u, int fa) {

```

```

22     maxdep[u] = dep[u];
23     for (auto [w, v] : e[u]) {
24         if (v == fa) continue;
25         dep[v] = dep[u] + 1;
26         parent[v] = u;
27         dfs(v, u);
28         maxdep[u] = max(maxdep[u], maxdep[v]);
29     }
30 }
31
32 void dfs1(int u, int fa) {
33     for (auto [w, v] : e[u]) {
34         if (v == fa) continue;
35         dfs1(v, u);
36         int x = d1[v] + w;
37         if (x > d1[u]) {
38             d2[u] = d1[u], s2[u] = s1[u];
39             d1[u] = x, s1[u] = v;
40         } else if (x > d2[u]) {
41             d2[u] = x, s2[u] = v;
42         }
43     }
44 }
45 void dfs2(int u, int fa) {
46     for (auto [w, v] : e[u]) {
47         if (v == fa) continue;
48         if (s1[u] == v) {
49             up[v] = max(up[u], d2[u]) + w;
50         } else {
51             up[v] = max(up[u], d1[u]) + w;
52         }
53         dfs2(v, u);
54     }
55 }
56
57 int radius, center, diam;
58 void getCenter() {
59     center = 1; //中心
60     for (int i = 1; i <= n; i++) {
61         if (max(d1[i], up[i]) < max(d1[center], up[center])) {
62             center = i;
63         }
64     }
65     radius = max(d1[center], up[center]); //距离最远点的距离的最小值
66     diam = d1[center] + up[center] + 1; //直径
67 }
68
69 int rem; //删除重心后剩余连通块体积的最小值
70 int cog; //重心
71 vector<bool> vis;
72 void getCog() {
73     vis.resize(n);
74     rem = INT_MAX;
75     cog = 1;
76     dfsCog(1);
77 }
78 int dfsCog(int u) {
79     vis[u] = true;
80     int s = 1, res = 0;
81     for (auto [w, v] : e[u]) {
82         if (vis[v]) continue;
83         int t = dfsCog(v);
84         res = max(res, t);
85         s += t;

```

```

86     }
87     res = max(res, n - s);
88     if (res < rem) {
89         rem = res;
90         cog = u;
91     }
92     return s;
93 }
94 };

```

2.3 点分治 / 树的重心

重心的定义：删除树上的某一个点，会得到若干棵子树；删除某点后，得到的最大子树最小，这个点称为重心。我们假设某个点是重心，记录此时最大子树的最小值，遍历完所有点后取最大值即可。

重心的性质：重心最多可能会有两个，且此时两个重心相邻。

点分治的一般过程是：取重心为新树的根，随后使用 `dfs` 处理当前这棵树，灵活运用 `child` 和 `pre` 两个数组分别计算通过根节点、不通过根节点的路径信息，根据需要进行答案的更新；再对子树分治，寻找子树的重心，……。时间复杂度降至 $\mathcal{O}(N \log N)$ 。

```

1  int root = 0, MaxTree = 1e18; //分别代表重心下标、最大子树大小
2  vector<int> vis(n + 1), siz(n + 1);
3  auto get = [&](auto self, int x, int fa, int n) -> void { // 获取树的重心
4      siz[x] = 1;
5      int val = 0;
6      for (auto [y, w] : ver[x]) {
7          if (y == fa || vis[y]) continue;
8          self(self, y, x, n);
9          siz[x] += siz[y];
10         val = max(val, siz[y]);
11     }
12     val = max(val, n - siz[x]);
13     if (val < MaxTree) {
14         MaxTree = val;
15         root = x;
16     }
17 };
18
19 auto clac = [&](int x) -> void { // 以 x 为新的根，维护询问
20     set<int> pre = {0}; // 记录到根节点 x 距离为 i 的路径是否存在
21     vector<int> dis(n + 1);
22     for (auto [y, w] : ver[x]) {
23         if (vis[y]) continue;
24         vector<int> child; // 记录 x 的子树节点的深度信息
25         auto dfs = [&](auto self, int x, int fa) -> void {
26             child.push_back(dis[x]);
27             for (auto [y, w] : ver[x]) {
28                 if (y == fa || vis[y]) continue;
29                 dis[y] = dis[x] + w;
30                 self(self, y, x);
31             }
32         };
33         dis[y] = w;
34         dfs(dfs, y, x);
35
36         for (auto it : child) {
37             for (int i = 1; i <= m; i++) { // 根据询问更新值
38                 if (q[i] < it || !pre.count(q[i] - it)) continue;
39                 ans[i] = 1;
40             }
41         }

```

```

42     pre.insert(child.begin(), child.end());
43 }
44 };
45
46 auto dfz = [&](auto self, int x, int fa) -> void { // 点分治
47     vis[x] = 1; // 标记已经被更新过的旧重心，确保只对子树分治
48     clac(x);
49     for (auto [y, w] : ver[x]) {
50         if (y == fa || vis[y]) continue;
51         MaxTree = 1e18;
52         get(get, y, x, siz[y]);
53         self(self, root, x);
54     }
55 };
56
57 get(get, 1, 0, n);
58 dfz(dfz, root, 0);

```

2.4 最近公共祖先 LCA

2.4.1 树链剖分解法

预处理时间复杂度 $\mathcal{O}(N)$ ；单次查询 $\mathcal{O}(\log N)$ ，常数较小。

```

1  struct HLD {
2      int n, idx;
3      vector<vector<int>> ver;
4      vector<int> siz, dep;
5      vector<int> top, son, parent;
6
7      HLD(int n) {
8          this->n = n;
9          ver.resize(n + 1);
10         siz.resize(n + 1);
11         dep.resize(n + 1);
12
13         top.resize(n + 1);
14         son.resize(n + 1);
15         parent.resize(n + 1);
16     }
17     void add(int x, int y) { // 建立双向边
18         ver[x].push_back(y);
19         ver[y].push_back(x);
20     }
21     void dfs1(int x) {
22         siz[x] = 1;
23         dep[x] = dep[parent[x]] + 1;
24         for (auto y : ver[x]) {
25             if (y == parent[x]) continue;
26             parent[y] = x;
27             dfs1(y);
28             siz[x] += siz[y];
29             if (siz[y] > siz[son[x]]) {
30                 son[x] = y;
31             }
32         }
33     }
34     void dfs2(int x, int up) {
35         top[x] = up;
36         if (son[x]) dfs2(son[x], up);
37         for (auto y : ver[x]) {
38             if (y == parent[x] || y == son[x]) continue;

```



```

39         dfs2(y, y);
40     }
41 }
42 int lca(int x, int y) {
43     while (top[x] != top[y]) {
44         if (dep[top[x]] > dep[top[y]]) {
45             x = parent[top[x]];
46         } else {
47             y = parent[top[y]];
48         }
49     }
50     return dep[x] < dep[y] ? x : y;
51 }
52 int clac(int x, int y) { // 查询两点间距离
53     return dep[x] + dep[y] - 2 * dep[lca(x, y)];
54 }
55 void work(int root = 1) { // 在此初始化
56     dfs1(root);
57     dfs2(root, root);
58 }
59 };

```

2.4.2 树上倍增解法

预处理时间复杂度 $\mathcal{O}(N \log N)$ ；单次查询 $\mathcal{O}(\log N)$ ，但是常数比树链剖分解法更大。

封装一：基础封装，针对无权图。

```

1  struct Tree {
2      int n;
3      vector<vector<int>> ver, val;
4      vector<int> lg, dep;
5      Tree(int n) {
6          this->n = n;
7          ver.resize(n + 1);
8          val.resize(n + 1, vector<int>(30));
9          lg.resize(n + 1);
10         dep.resize(n + 1);
11         for (int i = 1; i <= n; i++) { //预处理 log
12             lg[i] = lg[i - 1] + (1 << lg[i - 1] == i);
13         }
14     }
15     void add(int x, int y) { // 建立双向边
16         ver[x].push_back(y);
17         ver[y].push_back(x);
18     }
19     void dfs(int x, int fa) {
20         val[x][0] = fa; // 储存 x 的父节点
21         dep[x] = dep[fa] + 1;
22         for (int i = 1; i <= lg[dep[x]]; i++) {
23             val[x][i] = val[val[x][i - 1]][i - 1];
24         }
25         for (auto y : ver[x]) {
26             if (y == fa) continue;
27             dfs(y, x);
28         }
29     }
30     int lca(int x, int y) {
31         if (dep[x] < dep[y]) swap(x, y);
32         while (dep[x] > dep[y]) {
33             x = val[x][lg[dep[x]] - dep[y] - 1];
34         }

```

```

35     if (x == y) return x;
36     for (int k = lg[dep[x]] - 1; k >= 0; k--) {
37         if (val[x][k] == val[y][k]) continue;
38         x = val[x][k];
39         y = val[y][k];
40     }
41     return val[x][0];
42 }
43 int clac(int x, int y) { // 倍增查询两点间距离
44     return dep[x] + dep[y] - 2 * dep[lca(x, y)];
45 }
46 void work(int root = 1) { // 在此初始化
47     dfs(root, 0);
48 }
49 };

```

封装二：扩展封装，针对有权图，支持“倍增查询两点路径上的最大边权”功能。

```

1  struct Tree {
2      int n;
3      vector<vector<int>> val, Max;
4      vector<vector<pair<int, int>>> ver;
5      vector<int> lg, dep;
6      Tree(int n) {
7          this->n = n;
8          ver.resize(n + 1);
9          val.resize(n + 1, vector<int>(30));
10         Max.resize(n + 1, vector<int>(30));
11         lg.resize(n + 1);
12         dep.resize(n + 1);
13         for (int i = 1; i <= n; i++) { //预处理 log
14             lg[i] = lg[i - 1] + (1 << lg[i - 1] == i);
15         }
16     }
17     void add(int x, int y, int w) { // 建立双向边
18         ver[x].push_back({y, w});
19         ver[y].push_back({x, w});
20     }
21     void dfs(int x, int fa) {
22         val[x][0] = fa;
23         dep[x] = dep[fa] + 1;
24         for (int i = 1; i <= lg[dep[x]]; i++) {
25             val[x][i] = val[val[x][i - 1]][i - 1];
26             Max[x][i] = max(Max[x][i - 1], Max[val[x][i - 1]][i - 1]);
27         }
28         for (auto [y, w] : ver[x]) {
29             if (y == fa) continue;
30             Max[y][0] = w;
31             dfs(y, x);
32         }
33     }
34     int lca(int x, int y) {
35         if (dep[x] < dep[y]) swap(x, y);
36         while (dep[x] > dep[y]) {
37             x = val[x][lg[dep[x]] - dep[y] - 1];
38         }
39         if (x == y) return x;
40         for (int k = lg[dep[x]] - 1; k >= 0; k--) {
41             if (val[x][k] == val[y][k]) continue;
42             x = val[x][k];
43             y = val[y][k];
44         }
45         return val[x][0];

```

```

46     }
47     int clac(int x, int y) { // 倍增查询两点间距离
48         return dep[x] + dep[y] - 2 * dep[lca(x, y)];
49     }
50     int query(int x, int y) { // 倍增查询两点路径上的最大边权 (带权图)
51         auto get = [&](int x, int y) -> int {
52             int ans = 0;
53             if (x == y) return ans;
54             for (int i = lg[dep[x]]; i >= 0; i--) {
55                 if (dep[val[x][i]] > dep[y]) {
56                     ans = max(ans, Max[x][i]);
57                     x = val[x][i];
58                 }
59             }
60             ans = max(ans, Max[x][0]);
61             return ans;
62         };
63         int fa = lca(x, y);
64         return max(get(x, fa), get(y, fa));
65     }
66     void work(int root = 1) { // 在此初始化
67         dfs(root, 0);
68     }
69 };

```

2.5 树上路径交

计算两条路径的交点数量，直接载入任意 LCA 封装即可。

```

1  int intersection(int x, int y, int X, int Y) {
2      vector<int> t = {lca(x, X), lca(x, Y), lca(y, X), lca(y, Y)};
3      sort(t.begin(), t.end());
4      int r = lca(x, y), R = lca(X, Y);
5      if (dep[t[0]] < min(dep[r], dep[R]) || dep[t[2]] < max(dep[r], dep[R])) {
6          return 0;
7      }
8      return 1 + clac(t[2], t[3]);
9  }

```

2.6 树上启发式合并 (DSU on tree)

$\mathcal{O}(N \log N)$ 。

```

1  struct HLD {
2      vector<vector<int>> e;
3      vector<int> siz, son, cnt;
4      vector<LL> ans;
5      LL sum, Max;
6      int hson;
7      HLD(int n) {
8          e.resize(n + 1);
9          siz.resize(n + 1);
10         son.resize(n + 1);
11         ans.resize(n + 1);
12         cnt.resize(n + 1);
13         hson = 0;
14         sum = 0;
15         Max = 0;
16     }
17     void add(int u, int v) {
18         e[u].push_back(v);

```

3 图论

3.1 常见概念

oriented graph : 有向图

bidirectional edges : 双向边

平面图：若能将无向图 $G = (V, E)$ 画在平面上使得任意两条无重合顶点的边不相交，则称 G 是平面图。

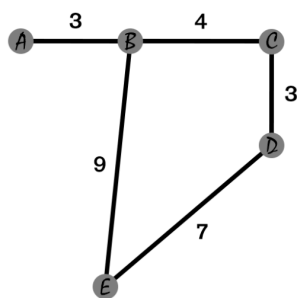
无向正权图上某一点的偏心距：记为 $ecc(u) = \max \{dist(u, v)\}$ ，即以这个点为源，到其他点的所有最短路的最大值。如下图 A 点， $ecc(A)$ 即为 12。

图的直径：定义为 $d = \max \{ecc(u)\}$ ，即最大的偏心距，亦可以简化为图中最远的一对点的距离。

图的中心：定义为 $arg = \min \{ecc(u)\}$ ，即偏心距最小的点。如下图，图的中心即为 B 点。

图的绝对中心：可以定义在边上的图的中心。

图的半径：图的半径不同于圆的半径，其不等于直径的一半（但对于绝对中心定义上的直径而言是一半）。定义为 $r = \min \{ecc(u)\}$ ，即中心的偏心距。计算方式：使用全源最短路，计算出所有点的偏心距，再加以计算。



3.2 单源最短路径 (SSSP问题)

3.2.1 (正权稀疏图) 动态数组存图+Dijkstra算法

使用优先队列优化，以 $\mathcal{O}(M \log N)$ 的复杂度计算。

```

1  vector<int> dis(n + 1, 1E18);
2  auto djikstra = [&](int s = 1) -> void {
3      using PII = pair<int, int>;
4      priority_queue<PII, vector<PII>, greater<PII>> q;
5      q.emplace(0, s);
6      dis[s] = 0;
7      vector<int> vis(n + 1);
8      while (!q.empty()) {
9          int x = q.top().second;
10         q.pop();
11         if (vis[x]) continue;
12         vis[x] = 1;
13         for (auto [y, w] : ver[x]) {
14             if (dis[y] > dis[x] + w) {
15                 dis[y] = dis[x] + w;
16                 q.emplace(dis[y], y);
17             }
18         }
19     }
20 }
  
```

```

17     }
18     }
19 }
20 };

```

3.2.2 (负权图、判负环) Bellman-ford 算法

使用结构体存边（该算法无需存图），以 $\mathcal{O}(NM)$ 的复杂度计算。

```

1  int n, m, s;
2  cin >> n >> m >> s;
3
4  vector<tuple<int, int, i64>> ver(m + 1);
5  for (int i = 1; i <= m; ++i) {
6      int x, y;
7      i64 w;
8      cin >> x >> y >> w;
9      ver[i] = {x, y, w};
10 }
11
12 vector<i64> dis(n + 1, inf), chk(n + 1);
13 dis[s] = 0;
14 for (int i = 1; i <= 2 * n; ++i) { // 双倍松弛，获取负环信息
15     vector<i64> backup = dis;
16     for (int j = 1; j <= m; ++j) {
17         auto [x, y, w] = ver[j];
18         chk[y] |= (i > n && backup[x] + w < dis[y]);
19         dis[y] = min(dis[y], backup[x] + w);
20     }
21 }
22
23 for (int i = 1; i <= n; ++i) {
24     if (i == s) {
25         cout << 0 << " ";
26     } else if (dis[i] >= inf / 2) {
27         cout << "no ";
28     } else if (chk[i]) {
29         cout << "inf ";
30     } else {
31         cout << dis[i] << " ";
32     }
33 }

```

3.2.3 (负权图) SPFA 算法

以 $\mathcal{O}(KM)$ 的复杂度计算，其中 K 虽然为常数，但是可以通过特殊的构造退化接近 N ，需要注意被卡。

```

1  const int N = 1e5 + 7, M = 1e6 + 7;
2  int n, m;
3  int ver[M], ne[M], h[N], edge[M], tot;
4  int d[N], v[N];
5
6  void add(int x, int y, int w) {
7      ver[++tot] = y, ne[tot] = h[x], h[x] = tot;
8      edge[tot] = w;
9  }
10 void spfa() {
11     ms(d, 0x3f); d[1] = 0;
12     queue<int> q; q.push(1);
13     v[1] = 1;

```

```

14     while(!q.empty()) {
15         int x = q.front(); q.pop(); v[x] = 0;
16         for (int i = h[x]; i; i = ne[i]) {
17             int y = ver[i];
18             if(d[y] > d[x] + edge[i]) {
19                 d[y] = d[x] + edge[i];
20                 if(v[y] == 0) q.push(y), v[y] = 1;
21             }
22         }
23     }
24 }
25 int main() {
26     cin >> n >> m;
27     for (int i = 1; i <= m; ++ i) {
28         int x, y, w; cin >> x >> y >> w;
29         add(x, y, w);
30     }
31     spfa();
32     for (int i = 1; i <= n; ++ i) {
33         if (d[i] == INF) cout << "N" << endl;
34         else cout << d[n] << endl;
35     }
36 }

```

3.3 多源汇最短路 (APSP问题)

使用邻接矩阵存图，可以处理负权边，以 $\mathcal{O}(N^3)$ 的复杂度计算。注意，这里建立的是单向边，计算双向边需要额外加边。

```

1  const int N = 210;
2  int n, m, d[N][N];
3
4  void floyd() {
5      for (int k = 1; k <= n; k++)
6          for (int i = 1; i <= n; i++)
7              for (int j = 1; j <= n; j++)
8                  d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
9  }
10 int main() {
11     cin >> n >> m;
12     for (int i = 1; i <= n; i++)
13         for (int j = 1; j <= n; j++)
14             if (i == j) d[i][j] = 0;
15             else d[i][j] = INF;
16     while (m--) {
17         int x, y, w; cin >> x >> y >> w;
18         d[x][y] = min(d[x][y], w);
19     }
20     floyd();
21     for (int i = 1; i <= n; ++ i) {
22         for (int j = 1; j <= n; ++ j) {
23             if (d[i][j] > INF / 2) cout << "N" << endl;
24             else cout << d[i][j] << endl;
25         }
26     }
27 }

```

3.4 平面图最短路（对偶图）

对于矩阵图，建立对偶图的过程如下（注释部分为建立原图），其中数据的给出顺序依次为：各 $n(n+1)$ 个数字分别代表从左向右、从上向下、从右向左、从下向上的边。

```

1  for (int i = 1; i <= n + 1; i++) {
2      for (int j = 1, w; j <= n; j++) {
3          cin >> w;
4          int pre = Hash(i - 1, j), now = Hash(i, j);
5          if (i == 1) {
6              add(s, now, w);
7          } else if (i == n + 1) {
8              add(pre, t, w);
9          } else {
10             add(pre, now, w);
11         }
12         // flow.add(Hash(i, j), Hash(i, j + 1), w);
13     }
14 }
15 for (int i = 1; i <= n; i++) {
16     for (int j = 1, w; j <= n + 1; j++) {
17         cin >> w;
18         int now = Hash(i, j), net = Hash(i, j - 1);
19         if (j == 1) {
20             add(now, t, w);
21         } else if (j == n + 1) {
22             add(s, net, w);
23         } else {
24             add(now, net, w);
25         }
26         // flow.add(Hash(i, j), Hash(i + 1, j), w);
27     }
28 }
29 for (int i = 1; i <= n + 1; i++) {
30     for (int j = 1, w; j <= n; j++) {
31         cin >> w;
32         int now = Hash(i, j), net = Hash(i - 1, j);
33         if (i == 1) {
34             add(now, s, w);
35         } else if (i == n + 1) {
36             add(t, net, w);
37         } else {
38             add(now, net, w);
39         }
40         // flow.add(Hash(i, j), Hash(i, j - 1), w);
41     }
42 }
43 for (int i = 1; i <= n; i++) {
44     for (int j = 1, w; j <= n + 1; j++) {
45         cin >> w;
46         int pre = Hash(i, j - 1), now = Hash(i, j);
47         if (j == 1) {
48             add(t, now, w);
49         } else if (j == n + 1) {
50             add(pre, s, w);
51         } else {
52             add(pre, now, w);
53         }
54         // flow.add(Hash(i, j), Hash(i - 1, j), w);
55     }
56 }

```

3.5 最小生成树 (MST问题)

3.5.1 (稀疏图) Prim算法

使用邻接矩阵存图，以 $\mathcal{O}(N^2 + M)$ 的复杂度计算，思想与 `dijkstra` 基本一致。

```

1  const int N = 550, INF = 0x3f3f3f3f;
2  int n, m, g[N][N];
3  int d[N], v[N];
4  int prim() {
5      ms(d, 0x3f); //这里的d表示到“最小生成树集合”的距离
6      int ans = 0;
7      for (int i = 0; i < n; ++ i) { //遍历 n 轮
8          int t = -1;
9          for (int j = 1; j <= n; ++ j)
10             if (v[j] == 0 && (t == -1 || d[j] < d[t])) //如果这个点不在集合内且当前距离
集合最近
11                 t = j;
12             v[t] = 1; //将t加入“最小生成树集合”
13             if (i && d[t] == INF) return INF; //如果发现不连通，直接返回
14             if (i) ans += d[t];
15             for (int j = 1; j <= n; ++ j) d[j] = min(d[j], g[t][j]); //用t更新其他点到集合
的距离
16         }
17         return ans;
18     }
19 int main() {
20     ms(g, 0x3f); cin >> n >> m;
21     while (m -- ) {
22         int x, y, w; cin >> x >> y >> w;
23         g[x][y] = g[y][x] = min(g[x][y], w);
24     }
25     int t = prim();
26     if (t == INF) cout << "impossible" << endl;
27     else cout << t << endl;
28 } //22.03.19已测试

```

3.5.2 (稠密图) Kruskal算法

平均时间复杂度为 $\mathcal{O}(M \log M)$ ，简化了并查集。

```

1  struct DSU {
2      vector<int> fa;
3      DSU(int n) : fa(n + 1) {
4          iota(fa.begin(), fa.end(), 0);
5      }
6      int get(int x) {
7          while (x != fa[x]) {
8              x = fa[x] = fa[fa[x]];
9          }
10         return x;
11     }
12     bool merge(int x, int y) { // 设x是y的祖先
13         x = get(x), y = get(y);
14         if (x == y) return false;
15         fa[y] = x;
16         return true;
17     }
18     bool same(int x, int y) {
19         return get(x) == get(y);
20     }
21 };

```



```

22 struct Tree {
23     using TII = tuple<int, int, int>;
24     int n;
25     priority_queue<TII, vector<TII>, greater<TII>> ver;
26
27     Tree(int n) {
28         this->n = n;
29     }
30     void add(int x, int y, int w) {
31         ver.emplace(w, x, y); // 注意顺序
32     }
33     int kruskal() {
34         DSU dsu(n);
35         int ans = 0, cnt = 0;
36         while (ver.size()) {
37             auto [w, x, y] = ver.top();
38             ver.pop();
39             if (dsu.same(x, y)) continue;
40             dsu.merge(x, y);
41             ans += w;
42             cnt++;
43         }
44         assert(cnt < n - 1); // 输入有误, 建树失败
45         return ans;
46     }
47 };

```

3.6 缩点 (Tarjan 算法)

3.6.1 (有向图) 强连通分量缩点

强连通分量缩点后的图称为 SCC。以 $\mathcal{O}(N + M)$ 的复杂度完成上述全部操作。

性质：缩点后的图拥有拓扑序 $color_{cnt}, color_{cnt-1}, \dots, 1$ ，可以不需再另跑一遍 topsort；缩点后的图是一张有向无环图（DAG、拓扑图）。

```

1 struct SCC {
2     int n, now, cnt;
3     vector<vector<int>> ver;
4     vector<int> dfn, low, col, S;
5
6     SCC(int n) : n(n), ver(n + 1), low(n + 1) {
7         dfn.resize(n + 1, -1);
8         col.resize(n + 1, -1);
9         now = cnt = 0;
10    }
11    void add(int x, int y) {
12        ver[x].push_back(y);
13    }
14    void tarjan(int x) {
15        dfn[x] = low[x] = now++;
16        S.push_back(x);
17        for (auto y : ver[x]) {
18            if (dfn[y] == -1) {
19                tarjan(y);
20                low[x] = min(low[x], low[y]);
21            } else if (col[y] == -1) {
22                low[x] = min(low[x], dfn[y]);
23            }
24        }
25        if (dfn[x] == low[x]) {
26            int pre;

```

5 数论

5.1 常见数列

5.1.1 调和级数

满足调和级数 $\mathcal{O}\left(\frac{N}{1} + \frac{N}{2} + \frac{N}{3} + \cdots + \frac{N}{N}\right)$ ，可以用 $\approx N \ln N$ 来拟合，但是会略小，误差量级在 10% 左右。本地可以在 500ms 内完成 10^8 量级的预处理计算。

N 的量级	1	2	3	4	5	6	7	8	9
累加和	27	482	7'069	93'668	1'166'750	13'970'034	162'725'364	1'857'511'568	20'877'697'634

下方示例为求解 1 到 N 中各个数字的因数值。

```

1  const int N = 1E5;
2  vector<vector<int>> dic(N + 1);
3  for (int i = 1; i <= N; i++) {
4      for (int j = i; j <= N; j += i) {
5          dic[j].push_back(i);
6      }
7  }
```

5.1.2 素数密度与分布

N 的量级	1	2	3	4	5	6	7	8	9
素数数量	4	25	168	1'229	9'592	78'498	664'579	5'761'455	50'847'534

除此之外，对于任意两个相邻的素数 $p_1, p_2 \leq 10^9$ ，有 $|p_1 - p_2| < 300$ 成立，更具体的说，最大的差值为 282。

5.1.3 因数最多数字与其因数数量

N 的量级	1	2	3	4	5	6	7
因数最多数字的因数数量	4	25	32	64	128	240	448
因数最多的数字	-	-	-	7560, 9240	83160, 98280	720720, 831600, 942480, 982800, 997920	-

5.2 快速幂

5.2.1 常规

```

1  i64 pow(i64 a, int b, int m) { // 复杂度是 log N
2      i64 r = 1 % m; /**! 这里的取模容易遗漏 */
3      for (; b >= 1, a = a * a % m) {
4          if (b & 1) r = r * a % m;
5      }
6      return r;
7  }
```

5.2.2 长整型 with 防爆乘法

```

1  i64 mul(i64 a, i64 b, i64 m) { // 由于不取模，运行速度非常快
2      a %= m, b %= m; /**! 这里的取模容易遗漏 */
3      i64 r = a * b - m * i64(1.L / m * a * b);
4      return r - m * (r >= m) + m * (r < 0);
5  }
6
7  i64 mul(i64 a, i64 b, i64 m) { // 比较慢
8      return (__int128)a * b % m;
9  }
10
11 i64 pow(i64 a, i64 b, i64 m) {
12     i64 res = 1 % m; /**! 这里的取模容易遗漏 */
13     for (; b; b >>= 1, a = mul(a, a, m)) { // 配合下方手写乘法
14         if (b & 1) res = mul(res, a, m);
15     }
16     return res;
17 }

```

5.3 质数判定

5.3.1 试除法

标准 $\mathcal{O}(\sqrt{N})$ ，有常数优化版本可达到 $\mathcal{O}(\frac{\sqrt{N}}{3})$ 。

```

1  bool isprime(int n) {
2      if (n < 2) return false;
3      for (int i = 2; i <= n / i; i++) {
4          if (n % i == 0) return false;
5      }
6      return true;
7  }

```

5.3.2 筛法

使用欧拉筛（线性筛），预期时间复杂度为 $\mathcal{O}(N)$ 。

```

1  vector<int> prime, minp;
2
3  void sieve(int n = 1e7) {
4      minp.resize(n + 1);
5      for (int i = 2; i <= n; i++) {
6          if (!minp[i]) {
7              minp[i] = i;
8              prime.push_back(i);
9          }
10         for (auto j : prime) {
11             if (j > minp[i] || j > n / i) break;
12             minp[i * j] = j;
13         }
14     }
15 }
16
17 bool isprime(int n) {
18     return minp[n] == n;
19 }

```

8 数据结构

8.1 并查集（全功能）

```

1 struct DSU {
2     vector<int> fa, p, e, f;
3
4     DSU(int n) {
5         fa.resize(n + 1);
6         iota(fa.begin(), fa.end(), 0);
7         p.resize(n + 1, 1);
8         e.resize(n + 1);
9         f.resize(n + 1);
10    }
11    int get(int x) {
12        while (x != fa[x]) {
13            x = fa[x] = fa[fa[x]];
14        }
15        return x;
16    }
17    bool merge(int x, int y) { // 设x是y的祖先
18        if (x == y) f[get(x)] = 1;
19        x = get(x), y = get(y);
20        e[x]++;
21        if (x == y) return false;
22        if (x < y) swap(x, y); // 将编号小的合并到大的上
23        fa[y] = x;
24        f[x] |= f[y], p[x] += p[y], e[x] += e[y];
25        return true;
26    }
27    bool same(int x, int y) {
28        return get(x) == get(y);
29    }
30    bool F(int x) { // 判断连通块内是否存在自环
31        return f[get(x)];
32    }
33    int size(int x) { // 输出连通块中点的数量
34        return p[get(x)];
35    }
36    int E(int x) { // 输出连通块中边的数量
37        return e[get(x)];
38    }
39 };
40

```

8.2 树状数组

8.2.1 半动态区间和（单点修改）

```

1 struct BIT {
2     vector<i64> w;
3     int n;
4     BIT(int n) : n(n), w(n + 1) {}
5     void add(int x, i64 v) {
6         for (; x <= n; x += x & -x) {
7             w[x] += v;
8         }
9     }
10    i64 rangeAsk(int l, int r) { // 差分实现区间和查询
11        auto ask = [&](int x) {
12            i64 ans = 0;

```

```

13         for (; x; x -= x & -x) {
14             ans += w[x];
15         }
16         return ans;
17     };
18     return ask(r) - ask(l - 1);
19 }
20 };

```

8.2.2 动态区间和（区间修改）

```

1 struct BIT {
2     vector<i64> a, b;
3     int n;
4
5     BIT(int n) : n(n), a(n + 1), b(n + 1) {}
6     void rangeAdd(int l, int r, i64 val) { // 区间修改
7         auto add = [&](int pos, i64 val) {
8             for (int i = pos; i <= n; i += i & -i) {
9                 a[i] += val;
10                b[i] += pos * val;
11            }
12        };
13        add(l, val), add(r + 1, -val);
14    }
15     i64 rangeSum(int l, int r) { // 区间和查询
16         auto sum = [&](int x) {
17             i64 ans = 0;
18             for (int i = x; i; i -= i & -i) {
19                 ans += (x + 1) * a[i] - b[i];
20             }
21             return ans;
22        };
23        return sum(r) - sum(l - 1);
24    }
25 };

```

8.2.3 静态区间最值（单点修改）

```

1 struct BIT {
2     vector<i64> w;
3     int n;
4     BIT(int n) : n(n), w(2 * n + 1) {}
5
6     void modify(int pos, i64 val) {
7         for (w[pos += n] = val; pos > 1; pos /= 2) {
8             w[pos / 2] = max(w[pos], w[pos ^ 1]);
9         }
10    }
11
12     i64 rangeMax(int l, int r) {
13         r++; // 使用开区间实现
14         i64 res = -1e18;
15         for (l += n, r += n; l < r; l /= 2, r /= 2) {
16             if (l % 2) res = max(res, w[l++]);
17             if (r % 2) res = max(res, w[--r]);
18         }
19         return res;
20    }
21 };

```

8.2.4 逆序对扩展

性质：交换序列的任意两元素，序列的逆序数的奇偶性必定发生改变。

```

1 struct BIT {
2     int n;
3     vector<int> w, chk; // chk 为传入的待处理数组
4     BIT(auto &in) : n(in.size() - 1), w(in.size()), chk(in) {}
5     void add(int x, i64 v) {...}
6     i64 rangeAsk(int l, int r) {...}
7     i64 get() {
8         vector<array<int, 2>> alls;
9         for (int i = 1; i <= n; i++) {
10             alls.push_back({chk[i], i});
11         }
12         sort(alls.begin(), alls.end());
13         i64 ans = 0;
14         for (auto [val, idx] : alls) {
15             ans += ask(idx + 1, n);
16             add(idx, 1);
17         }
18         return ans;
19     }
20 };

```

8.2.5 前驱后继扩展 (常规+第 k 小值查询+元素排名查询+元素前驱后继查询)

注意，被查询的值都应该小于等于 N ，否则会越界；如果离散化不可使用，则需要使用平衡树替代。

```

1 struct BIT {
2     int n;
3     vector<int> w;
4     BIT(int n) : n(n), w(n + 1) {}
5     void add(int x, int v) {
6         for (; x <= n; x += x & -x) {
7             w[x] += v;
8         }
9     }
10    int kth(int x) { // 查找第 k 小的值
11        int ans = 0;
12        for (int i = __lg(n); i >= 0; i--) {
13            int val = ans + (1 << i);
14            if (val < n && w[val] < x) {
15                x -= w[val];
16                ans = val;
17            }
18        }
19        return ans + 1;
20    }
21    int get(int x) { // 查找 x 的排名
22        int ans = 1;
23        for (x--; x; x -= x & -x) {
24            ans += w[x];
25        }
26        return ans;
27    }
28    int pre(int x) { return kth(get(x) - 1); } // 查找 x 的前驱
29    int suf(int x) { return kth(get(x + 1)); } // 查找 x 的后继
30 };

```

```

31 const int N = 10000000; // 可以用于在线处理平衡二叉树的全部要求
32 signed main() {
33     BIT bit(N + 1); // 在线处理不能够离散化，一定要开到比最大值更大
34     int n;
35     cin >> n;
36     for (int i = 1; i <= n; i++) {
37         int op, x;
38         cin >> op >> x;
39         if (op == 1) bit.add(x, 1); // 插入 x
40         else if (op == 2) bit.add(x, -1); // 删除任意一个 x
41         else if (op == 3) cout << bit.get(x) << "\n"; // 查询 x 的排名
42         else if (op == 4) cout << bit.kth(x) << "\n"; // 查询排名为 x 的数
43         else if (op == 5) cout << bit.pre(x) << "\n"; // 求小于 x 的最大值（前驱）
44         else if (op == 6) cout << bit.suf(x) << "\n"; // 求大于 x 的最小值（后继）
45     }
46 }

```

8.3 二维树状数组

8.3.1 半动态矩阵和（单点修改）

封装一：该版本不能同时进行区间修改+区间求和。空间占用为 $\mathcal{O}(NM)$ 、建树复杂度为 $\mathcal{O}(NM)$ 、单次查询复杂度为 $\mathcal{O}(\log N \cdot \log M)$ 。

```

1  template<class T> struct BIT_2D {
2      int n, m;
3      vector<vector<T>>> w;
4
5      BIT_2D(int n, int m) : n(n), m(m) {
6          w.resize(n + 1, vector<T>(m + 1));
7      }
8      void pointAdd(int x, int y, T k) {
9          for (int i = x; i <= n; i += i & -i) {
10             for (int j = y; j <= m; j += j & -j) {
11                 w[i][j] += k;
12             }
13         }
14     }
15     void matrixAdd(int x, int y, int X, int Y, T k) { // 区块修改：二维差分
16         X++, Y++;
17         pointAdd(x, y, k), pointAdd(X, y, -k);
18         pointAdd(X, Y, k), pointAdd(x, Y, -k);
19     }
20     T pointSum(int x, int y) {
21         T ans = T();
22         for (int i = x; i; i -= i & -i) {
23             for (int j = y; j; j -= j & -j) {
24                 ans += w[i][j];
25             }
26         }
27         return ans;
28     }
29     T matrixSum(int x, int y, int X, int Y) { // 区块查询：二维前缀和
30         x--, y--;
31         return pointSum(X, Y) - pointSum(x, Y) - pointSum(X, y) + pointSum(x, y);
32     }
33 };

```

8.3.2 动态矩阵和（区间修改）

封装二：仅支持区间修改+区间求和。但是时空复杂度均比上一个版本多 4 倍。

```

1  template<class T> struct BIT_2D {
2      int n, m;
3      vector<vector<T>> b1, b2, b3, b4;
4
5      BIT_2D(int n, int m) : n(n), m(m) {
6          b1.resize(n + 1, vector<T>(m + 1));
7          b2.resize(n + 1, vector<T>(m + 1));
8          b3.resize(n + 1, vector<T>(m + 1));
9          b4.resize(n + 1, vector<T>(m + 1));
10     }
11     void matrixAdd(int x, int y, int X, int Y, T k) { // 区块修改：二维差分
12         X++, Y++;
13         auto add = [&](int x, int y, T k) {
14             for (int i = x; i <= n; i += i & -i) {
15                 for (int j = y; j <= m; j += j & -j) {
16                     b1[i][j] += k;
17                     b2[i][j] += k * (x - 1);
18                     b3[i][j] += k * (y - 1);
19                     b4[i][j] += k * (x - 1) * (y - 1);
20                 }
21             }
22         };
23         add(x, y, k), add(X, y, -k);
24         add(X, Y, k), add(x, Y, -k);
25     }
26     T matrixSum(int x, int y, int X, int Y) { // 区块查询：二维前缀和
27         x--, y--;
28         auto ask = [&](int x, int y) {
29             T ans = T();
30             for (int i = x; i; i -= i & -i) {
31                 for (int j = y; j; j -= j & -j) {
32                     ans += x * y * b1[i][j];
33                     ans -= y * b2[i][j];
34                     ans -= x * b3[i][j];
35                     ans += b4[i][j];
36                 }
37             }
38             return ans;
39         };
40         return ask(X, Y) - ask(x, Y) - ask(X, y) + ask(x, y);
41     }
42 };

```

8.4 线段树

8.4.1 区间加法修改、区间最小值查询

```

1  template<class T> struct Segt {
2      struct node {
3          int l, r;
4          T w, rmq, lazy;
5      };
6      vector<T> w;
7      vector<node> t;
8
9      Segt() {}
10     Segt(int n) { init(n); }
11     Segt(vector<int> in) {

```



```

12     int n = in.size() - 1;
13     w.resize(n + 1);
14     for (int i = 1; i <= n; i++) {
15         w[i] = in[i];
16     }
17     init(in.size() - 1);
18 }
19
20 #define GL (k << 1)
21 #define GR (k << 1 | 1)
22
23 void init(int n) {
24     w.resize(n + 1);
25     t.resize(n * 4 + 1);
26     auto build = [&](auto self, int l, int r, int k = 1) {
27         if (l == r) {
28             t[k] = {l, r, w[l], w[l], -1}; // 如果有赋值为 0 的操作，则懒标记必须要
-1
29             return;
30         }
31         t[k] = {l, r, 0, 0, -1};
32         int mid = (l + r) / 2;
33         self(self, l, mid, GL);
34         self(self, mid + 1, r, GR);
35         pushup(k);
36     };
37     build(build, 1, n);
38 }
39 void pushdown(node &p, T lazy) { /* 【在此更新下递函数】 */
40     p.w += (p.r - p.l + 1) * lazy;
41     p.rmq += lazy;
42     p.lazy += lazy;
43 }
44 void pushdown(int k) {
45     if (t[k].lazy == -1) return;
46     pushdown(t[GL], t[k].lazy);
47     pushdown(t[GR], t[k].lazy);
48     t[k].lazy = -1;
49 }
50 void pushup(int k) {
51     auto pushup = [&](node &p, node &l, node &r) { /* 【在此更新上传函数】 */
52         p.w = l.w + r.w;
53         p.rmq = min(l.rmq, r.rmq); // RMQ -> min/max
54     };
55     pushup(t[k], t[GL], t[GR]);
56 }
57 void modify(int l, int r, T val, int k = 1) { // 区间修改
58     if (l <= t[k].l && t[k].r <= r) {
59         pushdown(t[k], val);
60         return;
61     }
62     pushdown(k);
63     int mid = (t[k].l + t[k].r) / 2;
64     if (l <= mid) modify(l, r, val, GL);
65     if (mid < r) modify(l, r, val, GR);
66     pushup(k);
67 }
68 T rmq(int l, int r, int k = 1) { // 区间询问最小值
69     if (l <= t[k].l && t[k].r <= r) {
70         return t[k].rmq;
71     }
72     pushdown(k);
73     int mid = (t[k].l + t[k].r) / 2;
74     T ans = numeric_limits<T>::max(); // RMQ -> 为 max 时需要修改为 ::lowest()

```

```

75     if (l <= mid) ans = min(ans, rmq(l, r, GL)); // RMQ -> min/max
76     if (mid < r) ans = min(ans, rmq(l, r, GR)); // RMQ -> min/max
77     return ans;
78 }
79 T ask(int l, int r, int k = 1) { // 区间询问
80     if (l <= t[k].l && t[k].r <= r) {
81         return t[k].w;
82     }
83     pushdown(k);
84     int mid = (t[k].l + t[k].r) / 2;
85     T ans = 0;
86     if (l <= mid) ans += ask(l, r, GL);
87     if (mid < r) ans += ask(l, r, GR);
88     return ans;
89 }
90 void debug(int k = 1) {
91     cout << "[" << t[k].l << ", " << t[k].r << "]: ";
92     cout << "w = " << t[k].w << ", ";
93     cout << "Min = " << t[k].rmq << ", ";
94     cout << "lazy = " << t[k].lazy << ", ";
95     cout << endl;
96     if (t[k].l == t[k].r) return;
97     debug(GL), debug(GR);
98 }
99 };

```

8.4.2 同时需要处理区间加法与乘法修改

```

1  template <class T> struct Segt_ {
2      struct node {
3          int l, r;
4          T w, add, mul = 1; // 注意初始赋值
5      };
6      vector<T> w;
7      vector<node> t;
8
9      Segt_(int n) {
10         w.resize(n + 1);
11         t.resize((n << 2) + 1);
12         build(1, n);
13     }
14     Segt_(vector<int> in) {
15         int n = in.size() - 1;
16         w.resize(n + 1);
17         for (int i = 1; i <= n; i++) {
18             w[i] = in[i];
19         }
20         t.resize((n << 2) + 1);
21         build(1, n);
22     }
23     void pushdown(node &p, T add, T mul) { // 在此更新下递函数
24         p.w = p.w * mul + (p.r - p.l + 1) * add;
25         p.add = p.add * mul + add;
26         p.mul *= mul;
27     }
28     void pushup(node &p, node &l, node &r) { // 在此更新上传函数
29         p.w = l.w + r.w;
30     }
31     #define GL (k << 1)
32     #define GR (k << 1 | 1)
33     void pushdown(int k) { // 不需要动
34         pushdown(t[GL], t[k].add, t[k].mul);
35         pushdown(t[GR], t[k].add, t[k].mul);

```

```

36     t[k].add = 0, t[k].mul = 1;
37 }
38 void pushup(int k) { // 不需要动
39     pushup(t[k], t[GL], t[GR]);
40 }
41 void build(int l, int r, int k = 1) {
42     if (l == r) {
43         t[k] = {l, r, w[l]};
44         return;
45     }
46     t[k] = {l, r};
47     int mid = (l + r) / 2;
48     build(l, mid, GL);
49     build(mid + 1, r, GR);
50     pushup(k);
51 }
52 void modify(int l, int r, T val, int k = 1) { // 区间修改
53     if (l <= t[k].l && t[k].r <= r) {
54         t[k].w += (t[k].r - t[k].l + 1) * val;
55         t[k].add += val;
56         return;
57     }
58     pushdown(k);
59     int mid = (t[k].l + t[k].r) / 2;
60     if (l <= mid) modify(l, r, val, GL);
61     if (mid < r) modify(l, r, val, GR);
62     pushup(k);
63 }
64 void modify2(int l, int r, T val, int k = 1) { // 区间修改
65     if (l <= t[k].l && t[k].r <= r) {
66         t[k].w *= val;
67         t[k].add *= val;
68         t[k].mul *= val;
69         return;
70     }
71     pushdown(k);
72     int mid = (t[k].l + t[k].r) / 2;
73     if (l <= mid) modify2(l, r, val, GL);
74     if (mid < r) modify2(l, r, val, GR);
75     pushup(k);
76 }
77 T ask(int l, int r, int k = 1) { // 区间询问, 不合并
78     if (l <= t[k].l && t[k].r <= r) {
79         return t[k].w;
80     }
81     pushdown(k);
82     int mid = (t[k].l + t[k].r) / 2;
83     T ans = 0;
84     if (l <= mid) ans += ask(l, r, GL);
85     if (mid < r) ans += ask(l, r, GR);
86     return ans;
87 }
88 void debug(int k = 1) {
89     cout << "[" << t[k].l << ", " << t[k].r << "]: ";
90     cout << "w = " << t[k].w << ", ";
91     cout << "add = " << t[k].add << ", ";
92     cout << "mul = " << t[k].mul << ", ";
93     cout << endl;
94     if (t[k].l == t[k].r) return;
95     debug(GL), debug(GR);
96 }
97 };

```

8.4.3 区间赋值/推平

如果存在推平为 0 的操作，那么需要将 lazy 初始赋值为 -1。

```

1 void pushdown(node &p, T lazy) { /* 【在此更新下递函数】 */
2     p.w = (p.r - p.l + 1) * lazy;
3     p.lazy = lazy;
4 }
5 void modify(int l, int r, T val, int k = 1) {
6     if (l <= t[k].l && t[k].r <= r) {
7         t[k].w = val;
8         t[k].lazy = val;
9         return;
10    }
11    // 剩余部分不变
12 }

```

8.4.4 区间取模

原题需要进行“单点赋值+区间取模+区间求和” [See](#)。该操作不需要懒标记。

需要额外维护一个区间最大值，当模数大于区间最大值时剪枝，否则进行单点取模。由于单点 $\text{MOD} < x$ 时 $x \bmod \text{MOD} < \frac{x}{2}$ ，故单点取模至 0 最劣只需要 $\log x$ 次。

```

1 void modifyMod(int l, int r, T val, int k = 1) {
2     if (l <= t[k].l && t[k].r <= r) {
3         if (t[k].rmq < val) return; // 重要剪枝
4     }
5     if (t[k].l == t[k].r) {
6         t[k].w %= val;
7         t[k].rmq %= val;
8         return;
9     }
10    int mid = (t[k].l + t[k].r) / 2;
11    if (l <= mid) modifyMod(l, r, val, GL);
12    if (mid < r) modifyMod(l, r, val, GR);
13    pushup(k);
14 }

```

8.4.5 区间异或修改

原题需要维护“区间异或修改+区间求和” [See](#)。

```

1 struct Segt { // #define GL (k << 1) // #define GR (k << 1 | 1)
2     struct node {
3         int l, r;
4         int w[N], lazy; // 注意这里为了方便计算, w 只需要存位
5     };
6     vector<int> base;
7     vector<node> t;
8
9     Segt(vector<int> in) : base(in) {
10        int n = in.size() - 1;
11        t.resize(n * 4 + 1);
12        auto build = [&](auto self, int l, int r, int k = 1) {
13            t[k] = {l, r}; // 前置赋值
14            if (l == r) {
15                for (int i = 0; i < N; i++) {
16                    t[k].w[i] = base[l] >> i & 1;
17                }
18            }
19        };
20        build(self, 1, n + 1, 1);
21    }
22
23    int query(int l, int r) {
24        int ans = 0;
25        auto dfs = [&](auto self, int l, int r, int k) {
26            if (l <= t[k].l && t[k].r <= r) {
27                ans ^= t[k].w[t[k].rmq];
28            }
29            int mid = (t[k].l + t[k].r) / 2;
30            if (l <= mid) dfs(self, l, r, GL);
31            if (mid < r) dfs(self, l, r, GR);
32        };
33        dfs(self, l, r, 1);
34        return ans;
35    }
36
37    void modify(int l, int r, int val) {
38        auto dfs = [&](auto self, int l, int r, int k) {
39            if (l <= t[k].l && t[k].r <= r) {
40                t[k].w[t[k].rmq] ^= val;
41            }
42            int mid = (t[k].l + t[k].r) / 2;
43            if (l <= mid) dfs(self, l, r, GL);
44            if (mid < r) dfs(self, l, r, GR);
45            pushup(k);
46        };
47        dfs(self, l, r, 1);
48    }
49
50    void pushup(int k) {
51        int l = t[k].l, r = t[k].r;
52        t[k].w[t[k].rmq] = t[t[k].GL].w[t[t[k].GL].rmq] ^ t[t[k].GR].w[t[t[k].GR].rmq];
53    }
54
55    void pushdown(int k) {
56        int l = t[k].l, r = t[k].r;
57        int mid = (l + r) / 2;
58        t[t[k].GL].lazy ^= t[k].w[t[k].rmq];
59        t[t[k].GR].lazy ^= t[k].w[t[k].rmq];
60        t[t[k].GL].rmq = t[t[k].GL].l;
61        t[t[k].GR].rmq = t[t[k].GR].l;
62    }
63
64    int rmq(int k) {
65        return t[k].rmq;
66    }
67
68    void setrmq(int k, int val) {
69        t[k].rmq = val;
70    }
71
72    void setrmq(int k, int val, int GL, int GR) {
73        t[k].rmq = val;
74        t[t[k].GL].rmq = GL;
75        t[t[k].GR].rmq = GR;
76    }
77
78    void setrmq(int k, int val, int GL, int GR, int GL2, int GR2) {
79        t[k].rmq = val;
80        t[t[k].GL].rmq = GL;
81        t[t[k].GR].rmq = GR;
82        t[t[k].GL2].rmq = GL2;
83        t[t[k].GR2].rmq = GR2;
84    }
85
86    void setrmq(int k, int val, int GL, int GR, int GL2, int GR2, int GL3, int GR3) {
87        t[k].rmq = val;
88        t[t[k].GL].rmq = GL;
89        t[t[k].GR].rmq = GR;
90        t[t[k].GL2].rmq = GL2;
91        t[t[k].GR2].rmq = GR2;
92        t[t[k].GL3].rmq = GL3;
93        t[t[k].GR3].rmq = GR3;
94    }
95
96    void setrmq(int k, int val, int GL, int GR, int GL2, int GR2, int GL3, int GR3, int GL4, int GR4) {
97        t[k].rmq = val;
98        t[t[k].GL].rmq = GL;
99        t[t[k].GR].rmq = GR;
100       t[t[k].GL2].rmq = GL2;
101       t[t[k].GR2].rmq = GR2;
102       t[t[k].GL3].rmq = GL3;
103       t[t[k].GR3].rmq = GR3;
104       t[t[k].GL4].rmq = GL4;
105       t[t[k].GR4].rmq = GR4;
106    }
107
108    void setrmq(int k, int val, int GL, int GR, int GL2, int GR2, int GL3, int GR3, int GL4, int GR4, int GL5, int GR5) {
109        t[k].rmq = val;
110        t[t[k].GL].rmq = GL;
111        t[t[k].GR].rmq = GR;
112        t[t[k].GL2].rmq = GL2;
113        t[t[k].GR2].rmq = GR2;
114        t[t[k].GL3].rmq = GL3;
115        t[t[k].GR3].rmq = GR3;
116        t[t[k].GL4].rmq = GL4;
117        t[t[k].GR4].rmq = GR4;
118        t[t[k].GL5].rmq = GL5;
119        t[t[k].GR5].rmq = GR5;
120    }
121
122    void setrmq(int k, int val, int GL, int GR, int GL2, int GR2, int GL3, int GR3, int GL4, int GR4, int GL5, int GR5, int GL6, int GR6) {
123        t[k].rmq = val;
124        t[t[k].GL].rmq = GL;
125        t[t[k].GR].rmq = GR;
126        t[t[k].GL2].rmq = GL2;
127        t[t[k].GR2].rmq = GR2;
128        t[t[k].GL3].rmq = GL3;
129        t[t[k].GR3].rmq = GR3;
130        t[t[k].GL4].rmq = GL4;
131        t[t[k].GR4].rmq = GR4;
132        t[t[k].GL5].rmq = GL5;
133        t[t[k].GR5].rmq = GR5;
134        t[t[k].GL6].rmq = GL6;
135        t[t[k].GR6].rmq = GR6;
136    }
137
138    void setrmq(int k, int val, int GL, int GR, int GL2, int GR2, int GL3, int GR3, int GL4, int GR4, int GL5, int GR5, int GL6, int GR6, int GL7, int GR7) {
139        t[k].rmq = val;
140        t[t[k].GL].rmq = GL;
141        t[t[k].GR].rmq = GR;
142        t[t[k].GL2].rmq = GL2;
143        t[t[k].GR2].rmq = GR2;
144        t[t[k].GL3].rmq = GL3;
145        t[t[k].GR3].rmq = GR3;
146        t[t[k].GL4].rmq = GL4;
147        t[t[k].GR4].rmq = GR4;
148        t[t[k].GL5].rmq = GL5;
149        t[t[k].GR5].rmq = GR5;
150        t[t[k].GL6].rmq = GL6;
151        t[t[k].GR6].rmq = GR6;
152        t[t[k].GL7].rmq = GL7;
153        t[t[k].GR7].rmq = GR7;
154    }
155
156    void setrmq(int k, int val, int GL, int GR, int GL2, int GR2, int GL3, int GR3, int GL4, int GR4, int GL5, int GR5, int GL6, int GR6, int GL7, int GR7, int GL8, int GR8) {
157        t[k].rmq = val;
158        t[t[k].GL].rmq = GL;
159        t[t[k].GR].rmq = GR;
160        t[t[k].GL2].rmq = GL2;
161        t[t[k].GR2].rmq = GR2;
162        t[t[k].GL3].rmq = GL3;
163        t[t[k].GR3].rmq = GR3;
164        t[t[k].GL4].rmq = GL4;
165        t[t[k].GR4].rmq = GR4;
166        t[t[k].GL5].rmq = GL5;
167        t[t[k].GR5].rmq = GR5;
168        t[t[k].GL6].rmq = GL6;
169        t[t[k].GR6].rmq = GR6;
170        t[t[k].GL7].rmq = GL7;
171        t[t[k].GR7].rmq = GR7;
172        t[t[k].GL8].rmq = GL8;
173        t[t[k].GR8].rmq = GR8;
174    }
175
176    void setrmq(int k, int val, int GL, int GR, int GL2, int GR2, int GL3, int GR3, int GL4, int GR4, int GL5, int GR5, int GL6, int GR6, int GL7, int GR7, int GL8, int GR8, int GL9, int GR9) {
177        t[k].rmq = val;
178        t[t[k].GL].rmq = GL;
179        t[t[k].GR].rmq = GR;
180        t[t[k].GL2].rmq = GL2;
181        t[t[k].GR2].rmq = GR2;
182        t[t[k].GL3].rmq = GL3;
183        t[t[k].GR3].rmq = GR3;
184        t[t[k].GL4].rmq = GL4;
185        t[t[k].GR4].rmq = GR4;
186        t[t[k].GL5].rmq = GL5;
187        t[t[k].GR5].rmq = GR5;
188        t[t[k].GL6].rmq = GL6;
189        t[t[k].GR6].rmq = GR6;
190        t[t[k].GL7].rmq = GL7;
191        t[t[k].GR7].rmq = GR7;
192        t[t[k].GL8].rmq = GL8;
193        t[t[k].GR8].rmq = GR8;
194        t[t[k].GL9].rmq = GL9;
195        t[t[k].GR9].rmq = GR9;
196    }
197
198    void setrmq(int k, int val, int GL, int GR, int GL2, int GR2, int GL3, int GR3, int GL4, int GR4, int GL5, int GR5, int GL6, int GR6, int GL7, int GR7, int GL8, int GR8, int GL9, int GR9, int GL10, int GR10) {
199        t[k].rmq = val;
200        t[t[k].GL].rmq = GL;
201        t[t[k].GR].rmq = GR;
202        t[t[k].GL2].rmq = GL2;
203        t[t[k].GR2].rmq = GR2;
204        t[t[k].GL3].rmq = GL3;
205        t[t[k].GR3].rmq = GR3;
206        t[t[k].GL4].rmq = GL4;
207        t[t[k].GR4].rmq = GR4;
208        t[t[k].GL5].rmq = GL5;
209        t[t[k].GR5].rmq = GR5;
210        t[t[k].GL6].rmq = GL6;
211        t[t[k].GR6].rmq = GR6;
212        t[t[k].GL7].rmq = GL7;
213        t[t[k].GR7].rmq = GR7;
214        t[t[k].GL8].rmq = GL8;
215        t[t[k].GR8].rmq = GR8;
216        t[t[k].GL9].rmq = GL9;
217        t[t[k].GR9].rmq = GR9;
218        t[t[k].GL10].rmq = GL10;
219        t[t[k].GR10].rmq = GR10;
220    }
221
222    void setrmq(int k, int val, int GL, int GR, int GL2, int GR2, int GL3, int GR3, int GL4, int GR4, int GL5, int GR5, int GL6, int GR6, int GL7, int GR7, int GL8, int GR8, int GL9, int GR9, int GL10, int GR10, int GL11, int GR11) {
223        t[k].rmq = val;
224        t[t[k].GL].rmq = GL;
225        t[t[k].GR].rmq = GR;
226        t[t[k].GL2].rmq = GL2;
227        t[t[k].GR2].rmq = GR2;
228        t[t[k].GL3].rmq = GL3;
229        t[t[k].GR3].rmq = GR3;
230        t[t[k].GL4].rmq = GL4;
231        t[t[k].GR4].rmq = GR4;
232        t[t[k].GL5].rmq = GL5;
233        t[t[k].GR5].rmq = GR5;
234        t[t[k].GL6].rmq = GL6;
235        t[t[k].GR6].rmq = GR6;
236        t[t[k].GL7].rmq = GL7;
237        t[t[k].GR7].rmq = GR7;
238        t[t[k].GL8].rmq = GL8;
239        t[t[k].GR8].rmq = GR8;
240        t[t[k].GL9].rmq = GL9;
241        t[t[k].GR9].rmq = GR9;
242        t[t[k].GL10].rmq = GL10;
243        t[t[k].GR10].rmq = GR10;
244        t[t[k].GL11].rmq = GL11;
245        t[t[k].GR11].rmq = GR11;
246    }
247
248    void setrmq(int k, int val, int GL, int GR, int GL2, int GR2, int GL3, int GR3, int GL4, int GR4, int GL5, int GR5, int GL6, int GR6, int GL7, int GR7, int GL8, int GR8, int GL9, int GR9, int GL10, int GR10, int GL11, int GR11, int GL12, int GR12) {
249        t[k].rmq = val;
250        t[t[k].GL].rmq = GL;
251        t[t[k].GR].rmq = GR;
252        t[t[k].GL2].rmq = GL2;
253        t[t[k].GR2].rmq = GR2;
254        t[t[k].GL3].rmq = GL3;
255        t[t[k].GR3].rmq = GR3;
256        t[t[k].GL4].rmq = GL4;
257        t[t[k].GR4].rmq = GR4;
258        t[t[k].GL5].rmq = GL5;
259        t[t[k].GR5].rmq = GR5;
260        t[t[k].GL6].rmq = GL6;
261        t[t[k].GR6].rmq = GR6;
262        t[t[k].GL7].rmq = GL7;
263        t[t[k].GR7].rmq = GR7;
264        t[t[k].GL8].rmq = GL8;
265        t[t[k].GR8].rmq = GR8;
266        t[t[k].GL9].rmq = GL9;
267        t[t[k].GR9].rmq = GR9;
268        t[t[k].GL10].rmq = GL10;
269        t[t[k].GR10].rmq = GR10;
270        t[t[k].GL11].rmq = GL11;
271        t[t[k].GR11].rmq = GR11;
272        t[t[k].GL12].rmq = GL12;
273        t[t[k].GR12].rmq = GR12;
274    }
275
276    void setrmq(int k, int val, int GL, int GR, int GL2, int GR2, int GL3, int GR3, int GL4, int GR4, int GL5, int GR5, int GL6, int GR6, int GL7, int GR7, int GL8, int GR8, int GL9, int GR9, int GL10, int GR10, int GL11, int GR11, int GL12, int GR12, int GL13, int GR13) {
277        t[k].rmq = val;
278        t[t[k].GL].rmq = GL;
279        t[t[k].GR].rmq = GR;
280        t[t[k].GL2].rmq = GL2;
281        t[t[k].GR2].rmq = GR2;
282        t[t[k].GL3].rmq = GL3;
283        t[t[k].GR3].rmq = GR3;
284        t[t[k].GL4].rmq = GL4;
285        t[t[k].GR4].rmq = GR4;
286        t[t[k].GL5].rmq = GL5;
287        t[t[k].GR5].rmq = GR5;
288        t[t[k].GL6].rmq = GL6;
289        t[t[k].GR6].rmq = GR6;
290        t[t[k].GL7].rmq = GL7;
291        t[t[k].GR7].rmq = GR7;
292        t[t[k].GL8].rmq = GL8;
293        t[t[k].GR8].rmq = GR8;
294        t[t[k].GL9].rmq = GL9;
295        t[t[k].GR9].rmq = GR9;
296        t[t[k].GL10].rmq = GL10;
297        t[t[k].GR10].rmq = GR10;
298        t[t[k].GL11].rmq = GL11;
299        t[t[k].GR11].rmq = GR11;
300        t[t[k].GL12].rmq = GL12;
301        t[t[k].GR12].rmq = GR12;
302        t[t[k].GL13].rmq = GL13;
303        t[t[k].GR13].rmq = GR13;
304    }
305
306    void setrmq(int k, int val, int GL, int GR, int GL2, int GR2, int GL3, int GR3, int GL4, int GR4, int GL5, int GR5, int GL6, int GR6, int GL7, int GR7, int GL8, int GR8, int GL9, int GR9, int GL10, int GR10, int GL11, int GR11, int GL12, int GR12, int GL13, int GR13, int GL14, int GR14) {
307        t[k].rmq = val;
308        t[t[k].GL].rmq = GL;
309        t[t[k].GR].rmq = GR;
310        t[t[k].GL2].rmq = GL2;
311        t[t[k].GR2].rmq = GR2;
312        t[t[k].GL3].rmq = GL3;
313        t[t[k].GR3].rmq = GR3;
314        t[t[k].GL4].rmq = GL4;
315        t[t[k].GR4].rmq = GR4;
316        t[t[k].GL5].rmq = GL5;
317        t[t[k].GR5].rmq = GR5;
318        t[t[k].GL6].rmq = GL6;
319        t[t[k].GR6].rmq = GR6;
320        t[t[k].GL7].rmq = GL7;
321        t[t[k].GR7].rmq = GR7;
322        t[t[k].GL8].rmq = GL8;
323        t[t[k].GR8].rmq = GR8;
324        t[t[k].GL9].rmq = GL9;
325        t[t[k].GR9].rmq = GR9;
326        t[t[k].GL10].rmq = GL10;
327        t[t[k].GR10].rmq = GR10;
328        t[t[k].GL11].rmq = GL11;
329        t[t[k].GR11].rmq = GR11;
330        t[t[k].GL12].rmq = GL12;
331        t[t[k].GR12].rmq = GR12;
332        t[t[k].GL13].rmq = GL13;
333        t[t[k].GR13].rmq = GR13;
334        t[t[k].GL14].rmq = GL14;
335        t[t[k].GR14].rmq = GR14;
336    }
337
338    void setrmq(int k, int val, int GL, int GR, int GL2, int GR2, int GL3, int GR3, int GL4, int GR4, int GL5, int GR5, int GL6, int GR6, int GL7, int GR7, int GL8, int GR8, int GL9, int GR9, int GL10, int GR10, int GL11, int GR11, int GL12, int GR12, int GL13, int GR13, int GL14, int GR14, int GL15, int GR15) {
339        t[k].rmq = val;
340        t[t[k].GL].rmq = GL;
341        t[t[k].GR].rmq = GR;
342        t[t[k].GL2].rmq = GL2;
343        t[t[k].GR2].rmq = GR2;
344        t[t[k].GL3].rmq = GL3;
345        t[t[k].GR3].rmq = GR3;
346        t[t[k].GL4].rmq = GL4;
347        t[t[k].GR4].rmq = GR4;
348        t[t[k].GL5].rmq = GL5;
349        t[t[k].GR5].rmq = GR5;
350        t[t[k].GL6].rmq = GL6;
351        t[t[k].GR6].rmq = GR6;
352        t[t[k].GL7].rmq = GL7;
353        t[t[k].GR7].rmq = GR7;
354        t[t[k].GL8].rmq = GL8;
355        t[t[k].GR8].rmq = GR8;
356        t[t[k].GL9].rmq = GL9;
357        t[t[k].GR9].rmq = GR9;
358        t[t[k].GL10].rmq = GL10;
359        t[t[k].GR10].rmq = GR10;
360        t[t[k].GL11].rmq = GL11;
361        t[t[k].GR11].rmq = GR11;
362        t[t[k].GL12].rmq = GL12;
363        t[t[k].GR12].rmq = GR12;
364        t[t[k].GL13].rmq = GL13;
365        t[t[k].GR13].rmq = GR13;
366        t[t[k].GL14].rmq = GL14;
367        t[t[k].GR14].rmq = GR14;
368        t[t[k].GL15].rmq = GL15;
369        t[t[k].GR15].rmq = GR15;
370    }
371
372    void setrmq(int k, int val, int GL, int GR, int GL2, int GR2, int GL3, int GR3, int GL4, int GR4, int GL5, int GR5, int GL6, int GR6, int GL7, int GR7, int GL8, int GR8, int GL9, int GR9, int GL10, int GR10, int GL11, int GR11, int GL12, int GR12, int GL13, int GR13, int GL14, int GR14, int GL15, int GR15, int GL16, int GR16) {
373        t[k].rmq = val;
374        t[t[k].GL].rmq = GL;
375        t[t[k].GR].rmq = GR;
376        t[t[k].GL2].rmq = GL2;
377        t[t[k].GR2].rmq = GR2;
378        t[t[k].GL3].rmq = GL3;
379        t[t[k].GR3].rmq = GR3;
380        t[t[k].GL4].rmq = GL4;
381        t[t[k].GR4].rmq = GR4;
382        t[t[k].GL5].rmq = GL5;
383        t[t[k].GR5].rmq = GR5;
384        t[t[k].GL6].rmq = GL6;
385        t[t[k].GR6].rmq = GR6;
386        t[t[k].GL7].rmq = GL7;
387        t[t[k].GR7].rmq = GR7;
388        t[t[k].GL8].rmq = GL8;
389        t[t[k].GR8].rmq = GR8;
390        t[t[k].GL9].rmq = GL9;
391        t[t[k].GR9].rmq = GR9;
392        t[t[k].GL10].rmq = GL10;
393        t[t[k].GR10].rmq = GR10;
394        t[t[k].GL11].rmq = GL11;
395        t[t[k].GR11].rmq = GR11;
396        t[t[k].GL12].rmq = GL12;
397        t[t[k].GR12].rmq = GR12;
398        t[t[k].GL13].rmq = GL13;
399        t[t[k].GR13].rmq = GR13;
400        t[t[k].GL14].rmq = GL14;
401        t[t[k].GR14].rmq = GR14;
402        t[t[k].GL15].rmq = GL15;
403        t[t[k].GR15].rmq = GR15;
404        t[t[k].GL16].rmq = GL16;
405        t[t[k].GR16].rmq = GR16;
406    }
407
408    void setrmq(int k, int val, int GL, int GR, int GL2, int GR2, int GL3, int GR3, int GL4, int GR4, int GL5, int GR5, int GL6, int GR6, int GL7, int GR7, int GL8, int GR8, int GL9, int GR9, int GL10, int GR10, int GL11, int GR11, int GL12, int GR12, int GL13, int GR13, int GL14, int GR14, int GL15, int GR15, int GL16, int GR16, int GL17, int GR17) {
409        t[k].rmq = val;
410        t[t[k].GL].rmq = GL;
411        t[t[k].GR].rmq = GR;
412        t[t[k].GL2].rmq = GL2;
413        t[t[k].GR2].rmq = GR2;
414        t[t[k].GL3].rmq = GL3;
415        t[t[k].GR3].rmq = GR3;
416        t[t[k].GL4].rmq = GL4;
417        t[t[k].GR4].rmq = GR4;
418        t[t[k].GL5].rmq = GL5;
419        t[t[k].GR5].rmq = GR5;
420        t[t[k].GL6].rmq = GL6;
421        t[t[k].GR6].rmq = GR6;
422        t[t[k].GL7].rmq = GL7;
423        t[t[k].GR7].rmq = GR7;
424        t[t[k].GL8].rmq = GL8;
425        t[t[k].GR8].rmq = GR8;
426        t[t[k].GL9].rmq = GL9;
427        t[t[k].GR9].rmq = GR9;
428        t[t[k].GL10].rmq = GL10;
429        t[t[k].GR10].rmq = GR10;
430        t[t[k].GL11].rmq = GL11;
431        t[t[k].GR11].rmq = GR11;
432        t[t[k].GL12].rmq = GL12;
433        t[t[k].GR12].rmq = GR12;
434        t[t[k].GL13].rmq = GL13;
435        t[t[k].GR13].rmq = GR13;
436        t[t[k].GL14].rmq = GL14;
437        t[t[k].GR14].rmq = GR14;
438        t[t[k].GL15].rmq = GL15;
439        t[t[k].GR15].rmq = GR15;
440        t[t[k].GL16].rmq = GL16;
441        t[t[k].GR16].rmq = GR16;
442        t[t[k].GL17].rmq = GL17;
443        t[t[k].GR17].rmq = GR17;
444    }
445
446    void setrmq(int k, int val, int GL, int GR, int GL2, int GR2, int GL3, int GR3, int GL4, int GR4, int GL5, int GR5, int GL6, int GR6, int GL7, int GR7, int GL8, int GR8, int GL9, int GR9, int GL10, int GR10, int GL11, int GR11, int GL12, int GR12, int GL13, int GR13, int GL14, int GR14, int GL15, int GR15, int GL16, int GR16, int GL17, int GR17, int GL18, int GR18) {
447        t[k].rmq = val;
448        t[t[k].GL].rmq = GL;
449        t[t[k].GR].rmq = GR;
450        t[t[k].GL2].rmq = GL2;
451        t[t[k].GR2].rmq = GR2;
452        t[t[k].GL3].rmq = GL3;
453        t[t[k].GR3].rmq = GR3;
454        t[t[k].GL4].rmq = GL4;
455        t[t[k].GR4].rmq = GR4;
456        t[t[k].GL5].rmq = GL5;
457        t[t[k].GR5].rmq = GR5;
458        t[t[k].GL6].rmq = GL6;
459        t[t[k].GR6].rmq = GR6;
460        t[t[k].GL7].rmq = GL7;
461        t[t[k].GR7].rmq = GR7;
462        t[t[k].GL8].rmq = GL8;
463        t[t[k].GR8].rmq = GR8;
464        t[t[k].GL9].rmq = GL9;
465        t[t[k].GR9].rmq = GR9;
466        t[t[k].GL10].rmq = GL10;
467        t[t[k].GR10].rmq = GR10;
468        t[t[k].GL11].rmq = GL11;
469        t[t[k].GR11].rmq = GR11;
470        t[t[k].GL12].rmq = GL12;
471        t[t[k].GR12].rmq = GR12;
472        t[t[k].GL13].rmq = GL13;
473        t[t[k].GR13].rmq = GR13;
474        t[t[k].GL14].rmq = GL14;
475        t[t[k].GR14].rmq = GR14;
476        t[t[k].GL15].rmq = GL15;
477        t[t[k].GR15].rmq = GR15;
478        t[t[k].GL16].rmq = GL16;
479        t[t[k].GR16].rmq = GR16;
480        t[t[k].GL17].rmq = GL17;
481        t[t[k].GR17].rmq = GR17;
482        t[t[k].GL18].rmq = GL18;
483        t[t[k].GR18].rmq = GR18;
484    }
485
486    void setrmq(int k, int val, int GL, int GR, int GL2, int GR2, int GL3, int GR3, int GL4, int GR4, int GL5, int GR5, int GL6, int GR6, int GL7, int GR7, int GL8, int GR8, int GL9, int GR9, int GL10, int GR10, int GL11, int GR11, int GL12, int GR12, int GL13, int GR13, int GL14, int GR14, int GL15, int GR15, int GL16, int GR16, int GL17, int GR17, int GL18, int GR18, int GL19, int GR19) {
487        t[k].rmq = val;
488        t[t[k].GL].rmq = GL;
489        t[t[k].GR].rmq = GR;
490        t[t[k].GL2].rmq = GL2;
491        t[t[k].GR2].rmq = GR2;
492        t[t[k].GL3].rmq = GL3;
493        t[t[k].GR3].rmq = GR3;
494        t[t[k].GL4].rmq = GL4;
495        t[t[k].GR4].rmq = GR4;
496        t[t[k].GL5].rmq = GL5;
497        t[t[k].GR5].rmq = GR5;
498        t[t[k].GL6].rmq = GL6;
499        t[t[k].GR6].rmq = GR6;
500        t[t[k].GL7].rmq = GL7;
501        t[t[k].GR7].rmq = GR7;
502        t[t[k].GL8].rmq = GL8;
503        t[t[k].GR8].rmq = GR8;
504        t[t[k].GL9].rmq = GL9;
505        t[t[k].GR9].rmq = GR9;
506        t[t[k].GL10].rmq = GL10;
507        t[t[k].GR10].rmq = GR10;
508        t[t[k].GL11].rmq = GL11;
509        t[t[k].GR11].rmq = GR11;
510        t[t[k].GL12].rmq = GL12;
511        t[t[k].GR12].rmq = GR12;
512        t[t[k].GL13].rmq = GL13;
513        t[t[k].GR13].rmq = GR13;
514        t[t[k].GL14].rmq = GL14;
515        t[t[k].GR14].rmq = GR14;
516        t[t[k].GL15].rmq = GL15;
517        t[t[k].GR15].rmq = GR15;
518        t[t[k].GL16].rmq = GL16;
519        t[t[k].GR16].rmq = GR16;
520        t[t[k].GL17].rmq = GL17;
521        t[t[k].GR17].rmq = GR17;
522        t[t[k].GL18].rmq = GL18;
523        t[t[k].GR18].rmq = GR18;
524        t[t[k].GL19].rmq = GL19;
525        t[t[k].GR19].rmq = GR19;
526    }
527
528    void setrmq(int k, int val, int GL, int GR, int GL2, int GR2, int GL3, int GR3, int GL4, int GR4, int GL5, int GR5, int GL6, int GR6, int GL7, int GR7, int GL8, int GR8, int GL9, int GR9, int GL10, int GR10, int GL11, int GR11, int GL12, int GR12, int GL13, int GR13, int GL14, int GR14, int GL15, int GR15, int GL16, int GR16, int GL17, int GR17, int GL18, int GR18, int GL19, int GR19, int GL20, int GR20) {
529        t[k].rmq = val;
530        t[t[k].GL].rmq = GL;
531        t[t[k].GR].rmq = GR;
532        t[t[k].GL2].rmq = GL2;
533        t[t[k].GR2].rmq = GR2;
534        t[t[k].GL3].rmq = GL3;
535        t[t[k].GR3].rmq = GR3;
536        t[t[k].GL4].rmq = GL4;
537        t[t[k].GR4].rmq = GR4;
538        t[t[k].GL5].rmq = GL5;
539        t[t[k].GR5].rmq = GR5;
540        t[t[k].GL6].rmq = GL6;
541        t[t[k].GR6].rmq = GR6;
542        t[t[k].GL7].rmq = GL7;
543        t[t[k].GR7].rmq = GR7;
544        t[t[k].GL8].rmq = GL8;
545        t[t[k].GR8].rmq = GR8;
546        t[t[k].GL9].rmq = GL9;
547        t[t[k].GR9].rmq = GR9;
548        t[t[k].GL10].rmq = GL10;
549        t[t[k].GR10].rmq = GR10;
550        t[t[k].GL11].rmq = GL11;
551        t[t[k].GR11].rmq = GR11;
552        t[t[k].GL12].rmq = GL12;
553        t[t[k].GR12].rmq = GR12;
554        t[t[k].GL13].rmq = GL13;
555        t[t[k].GR13].rmq = GR13;
556        t[t[k].GL14].rmq = GL14;
557        t[t[k].GR14].rmq = GR14;
558        t[t[k].GL15].rmq = GL15;
559        t[t[k].GR15].rmq = GR15;
560        t[t[k].GL16].rmq = GL16;
561        t[t[k].GR16].rmq = GR16;
562        t[t[k].GL17].rmq = GL17;
563        t[t[k].GR17].rmq = GR17;
564        t[t[k].GL18].rmq = GL18;
565        t[t[k].GR18].rmq = GR18;
566        t[t[k].GL19].rmq = GL19;
567        t[t[k].GR19].rmq = GR19;
568        t[t[k].GL20].rmq = GL20;
569        t[t[k].GR20].rmq = GR20;
570    }
571
572    void setrmq(int k, int val, int GL, int GR, int GL2, int GR2, int GL3, int GR3, int GL4, int GR4, int GL5, int GR5, int GL6, int GR6, int GL7, int GR7, int GL8, int GR8, int GL9, int GR9, int GL10, int GR10, int GL11, int GR11, int GL12, int GR12, int GL13, int GR13, int GL14, int GR14, int GL15, int GR15, int GL16, int GR16, int GL17, int GR17, int GL18, int GR18, int GL19, int GR19, int GL20, int GR20, int GL21, int GR21) {
5
```

```

18         return;
19     }
20     int mid = (l + r) / 2;
21     self(self, l, mid, GL);
22     self(self, mid + 1, r, GR);
23     pushup(k);
24 };
25 build(build, 1, n);
26 }
27 void pushdown(node &p, int lazy) { /* 【在此更新下递函数】 */
28     int len = p.r - p.l + 1;
29     for (int i = 0; i < N; i++) {
30         if (lazy >> i & 1) { // 即 p.w = (p.r - p.l + 1) - p.w;
31             p.w[i] = len - p.w[i];
32         }
33     }
34     p.lazy ^= lazy;
35 }
36 void pushdown(int k) { // 【不需要动】
37     if (t[k].lazy == 0) return;
38     pushdown(t[GL], t[k].lazy);
39     pushdown(t[GR], t[k].lazy);
40     t[k].lazy = 0;
41 }
42 void pushup(int k) {
43     auto pushup = [&](node &p, node &l, node &r) { /* 【在此更新上传函数】 */
44         for (int i = 0; i < N; i++) {
45             p.w[i] = l.w[i] + r.w[i]; // 即 p.w = l.w + r.w;
46         }
47     };
48     pushup(t[k], t[GL], t[GR]);
49 }
50 void modify(int l, int r, int val, int k = 1) { // 区间修改
51     if (l <= t[k].l && t[k].r <= r) {
52         pushdown(t[k], val);
53         return;
54     }
55     pushdown(k);
56     int mid = (t[k].l + t[k].r) / 2;
57     if (l <= mid) modify(l, r, val, GL);
58     if (mid < r) modify(l, r, val, GR);
59     pushup(k);
60 }
61 i64 ask(int l, int r, int k = 1) { // 区间求和
62     if (l <= t[k].l && t[k].r <= r) {
63         i64 ans = 0;
64         for (int i = 0; i < N; i++) {
65             ans += t[k].w[i] * (1LL << i);
66         }
67         return ans;
68     }
69     pushdown(k);
70     int mid = (t[k].l + t[k].r) / 2;
71     i64 ans = 0;
72     if (l <= mid) ans += ask(l, r, GL);
73     if (mid < r) ans += ask(l, r, GR);
74     return ans;
75 }
76 };

```

8.4.6 拆位运算

原题同上。使用若干棵线段树维护每一位的值，区间异或转变为区间翻转。

```

1  template<class T> struct Segt_ { // GL 为 (k << 1), GR 为 (k << 1 | 1)
2      struct node {
3          int l, r;
4          T w;
5          bool lazy; // 注意懒标记用布尔型足以
6      };
7      vector<T> w;
8      vector<node> t;
9
10     Segt_() {}
11     void init(vector<int> in) {
12         int n = in.size() - 1;
13         w.resize(n * 4 + 1);
14         for (int i = 0; i <= n; i++) { w[i] = in[i]; }
15         t.resize(n * 4 + 1);
16         build(1, n);
17     }
18     void pushdown(node &p, bool lazy = 1) { // 【在此更新下递函数】
19         p.w = (p.r - p.l + 1) - p.w;
20         p.lazy ^= lazy;
21     }
22     void pushup(node &p, node &l, node &r) { // 【在此更新上传函数】
23         p.w = l.w + r.w;
24     }
25     void pushdown(int k) { // 【不需要动】
26         if (t[k].lazy == 0) return;
27         pushdown(t[GL]), pushdown(t[GR]); // 注意这里不再需要传入第二个参数
28         t[k].lazy = 0;
29     }
30     void pushup(int k) { pushup(t[k], t[GL], t[GR]); } // 【不需要动】
31     void build(int l, int r, int k = 1) {
32         if (l == r) {
33             t[k] = {l, r, w[l], 0}; // 注意懒标记初始为 0
34             return;
35         }
36         t[k] = {l, r};
37         int mid = (l + r) / 2;
38         build(l, mid, GL);
39         build(mid + 1, r, GR);
40         pushup(k);
41     }
42     void reverse(int l, int r, int k = 1) { // 区间翻转
43         if (l <= t[k].l && t[k].r <= r) {
44             pushdown(t[k], 1);
45             return;
46         }
47         pushdown(k);
48         int mid = (t[k].l + t[k].r) / 2;
49         if (l <= mid) reverse(l, r, GL);
50         if (mid < r) reverse(l, r, GR);
51         pushup(k);
52     }
53     T ask(int l, int r, int k = 1) { // 区间求和
54         if (l <= t[k].l && t[k].r <= r) {
55             return t[k].w;
56         }
57         pushdown(k);
58         int mid = (t[k].l + t[k].r) / 2;
59         T ans = 0;

```

```

60         if (l <= mid) ans += ask(l, r, GL);
61         if (mid < r) ans += ask(l, r, GR);
62         return ans;
63     }
64 };
65 signed main() {
66     int n; cin >> n;
67     vector in(20, vector<int>(n + 1));
68     Segt_<i64> segt[20]; // 拆位建线段树
69     for (int i = 1, x; i <= n; i++) { cin >> x;
70         for (int bit = 0; bit < 20; bit++) {
71             in[bit][i] = x >> bit & 1;
72         }
73     }
74     for (int i = 0; i < 20; i++) {
75         segt[i].init(in[i]);
76     }
77
78     int m, op;
79     for (cin >> m; m; m--) { cin >> op;
80         if (op == 1) {
81             int l, r; i64 ans = 0; cin >> l >> r;
82             for (int i = 0; i < 20; i++) {
83                 ans += segt[i].ask(l, r) * (1LL << i);
84             }
85             cout << ans << "\n";
86         } else {
87             int l, r, val; cin >> l >> r >> val;
88             for (int i = 0; i < 20; i++) {
89                 if (val >> i & 1) { segt[i].reverse(l, r); }
90             }
91         }
92     }
93 }

```

8.5 坐标压缩与离散化

8.5.1 简单版本

```

1  sort(alls.begin(), alls.end());
2  alls.erase(unique(alls.begin(), alls.end()), alls.end());
3  auto get = [&](int x) {
4      return lower_bound(alls.begin(), alls.end(), x) - alls.begin();
5  };

```

8.5.2 封装

```

1  template <typename T> struct Compress_ {
2      int n, shift = 0; // shift 用于标记下标偏移量
3      vector<T> alls;
4
5      Compress_() {}
6      Compress_(auto in) : alls(in) {
7          init();
8      }
9      void add(T x) {
10         alls.emplace_back(x);
11     }
12     template <typename... Args> void add(T x, Args... args) {
13         add(x), add(args...);
14     }

```

```

15 void init() {
16     alls.emplace_back(numeric_limits<T>::max());
17     sort(alls.begin(), alls.end());
18     alls.erase(unique(alls.begin(), alls.end()), alls.end());
19     this->n = alls.size();
20 }
21 int size() {
22     return n;
23 }
24 int operator[](T x) { // 返回 x 元素的新下标
25     return upper_bound(alls.begin(), alls.end(), x) - alls.begin() + shift;
26 }
27 T Get(int x) { // 根据新下标返回原来元素
28     assert(x - shift < n);
29     return x - shift < n ? alls[x - shift] : -1;
30 }
31 bool count(T x) { // 查找元素 x 是否存在
32     return binary_search(alls.begin(), alls.end(), x);
33 }
34 friend auto &operator<< (ostream &o, const auto &j) {
35     cout << "{";
36     for (auto it : j.alls) {
37         o << it << " ";
38     }
39     return o << "}";
40 }
41 };
42 using Compress = Compress_<int>;

```

8.6 轻重链剖分/树链剖分

将线段树处理的部分分离，方便修改。支持链上查询/修改、子树查询/修改，建树时间复杂度 $\mathcal{O}(N \log N)$ ，单次查询时间复杂度 $\mathcal{O}(\log^2 N)$ 。

```

1 struct Segt {
2     struct node {
3         int l, r, w, lazy;
4     };
5     vector<int> w;
6     vector<node> t;
7
8     Segt() {}
9     #define GL (k << 1)
10    #define GR (k << 1 | 1)
11
12    void init(vector<int> in) {
13        int n = in.size() - 1;
14        w.resize(n + 1);
15        for (int i = 1; i <= n; i++) {
16            w[i] = in[i];
17        }
18        t.resize(n * 4 + 1);
19        auto build = [&](auto self, int l, int r, int k = 1) {
20            if (l == r) {
21                t[k] = {l, r, w[l], 0}; // 如果有赋值为 0 的操作，则懒标记必须要 -1
22                return;
23            }
24            t[k] = {l, r};
25            int mid = (l + r) / 2;
26            self(self, l, mid, GL);
27            self(self, mid + 1, r, GR);
28            pushup(k);

```



```

123         return;
124     }
125     if (in(p, a)) ans++;
126     query(ls[p], a);
127     query(rs[p], a);
128 }
129 int query(vector<int> &a) {
130     ans = 0;
131     query(rt, a);
132     return ans;
133 }
134 };

```

8.13 ST 表

用于解决区间可重复贡献问题，需要满足 x 运算符 $x = x$ （如区间最大值： $\max(x, x) = x$ 、区间 $\gcd$ ： $\gcd(x, x) = x$ 等），但是不支持修改操作。 $\mathcal{O}(N \log N)$ 预处理， $\mathcal{O}(1)$ 查询。

```

1  struct ST {
2      const int n, k;
3      vector<int> in1, in2;
4      vector<vector<int>> Max, Min;
5      ST(int n) : n(n), in1(n + 1), in2(n + 1), k(__lg(n)) {
6          Max.resize(k + 1, vector<int>(n + 1));
7          Min.resize(k + 1, vector<int>(n + 1));
8      }
9      void init() {
10         for (int i = 1; i <= n; i++) {
11             Max[0][i] = in1[i];
12             Min[0][i] = in2[i];
13         }
14         for (int i = 0, t = 1; i < k; i++, t <= 1) {
15             const int T = n - (t < 1) + 1;
16             for (int j = 1; j <= T; j++) {
17                 Max[i + 1][j] = max(Max[i][j], Max[i][j + t]);
18                 Min[i + 1][j] = min(Min[i][j], Min[i][j + t]);
19             }
20         }
21     }
22     int getMax(int l, int r) {
23         if (l > r) {
24             swap(l, r);
25         }
26         int k = __lg(r - l + 1);
27         return max(Max[k][l], Max[k][r - (1 < k) + 1]);
28     }
29     int getMin(int l, int r) {
30         if (l > r) {
31             swap(l, r);
32         }
33         int k = __lg(r - l + 1);
34         return min(Min[k][l], Min[k][r - (1 < k) + 1]);
35     }
36 };

```

```

60         l = l / B + 1;
61         r = r / B;
62         if (l < r) {
63             int k = __lg(r - l);
64             ans = min({ans, a[k][l], a[k][r - (1 << k)]}, cmp);
65         }
66         return ans;
67     } else {
68         int x = B * (l / B);
69         return ini[__builtin_ctzll(stk[r - 1] >> (l - x)) + 1];
70     }
71 }
72 };

```

8.15 pbds 扩展库实现平衡二叉树

记得加上相应的头文件，同时需要注意定义时的参数，一般只需要修改第三个参数：即定义的是大根堆还是小根堆。

附常见成员函数：

```

1 empty() / size()
2 insert(x) // 插入元素x
3 erase(x) // 删除元素/迭代器x
4 order_of_key(x) // 返回元素x的排名
5 find_by_order(x) // 返回排名为x的元素迭代器
6 lower_bound(x) / upper_bound(x) // 返回迭代器
7 join(Tree) // 将Tree树的全部元素并入当前的树
8 split(x, Tree) // 将大于x的元素放入Tree树

```

```

1 #include <ext/pb_ds/assoc_container.hpp>
2 using namespace __gnu_pbds;
3 using V = pair<int, int>;
4 tree<V, null_type, less<V>, rb_tree_tag, tree_order_statistics_node_update> ver;
5 map<int, int> dic;
6
7 int n; cin >> n;
8 for (int i = 1, op, x; i <= n; i++) {
9     cin >> op >> x;
10    if (op == 1) { // 插入一个元素x, 允许重复
11        ver.insert({x, ++dic[x]});
12    } else if (op == 2) { // 删除元素x, 若有重复, 则任意删除一个
13        ver.erase({x, dic[x]--});
14    } else if (op == 3) { // 查询元素x的排名 (排名定义为比当前数小的数的个数+1)
15        cout << ver.order_of_key({x, 1}) + 1 << endl;
16    } else if (op == 4) { // 查询排名为x的元素
17        cout << ver.find_by_order(--x)->first << endl;
18    } else if (op == 5) { // 查询元素x的前驱
19        int idx = ver.order_of_key({x, 1}) - 1; // 无论x存不存在, idx都代表x的位置, 需要-1
20        cout << ver.find_by_order(idx)->first << endl;
21    } else if (op == 6) { // 查询元素x的后继
22        int idx = ver.order_of_key({x, dic[x]}); // 如果x不存在, 那么idx就是x的后继
23        if (ver.find({x, 1}) != ver.end()) idx++; // 如果x存在, 那么idx是x的位置, 需要+1
24        cout << ver.find_by_order(idx)->first << endl;
25    }
26 }

```

8.16 vector 模拟实现平衡二叉树

```

1  #define ALL(x) x.begin(), x.end()
2  #define pre lower_bound
3  #define suf upper_bound
4  int n; cin >> n;
5  vector<int> ver;
6  for (int i = 1, op, x; i <= n; i++) {
7      cin >> op >> x;
8      if (op == 1) ver.insert(pre(ALL(ver), x), x);
9      if (op == 2) ver.erase(pre(ALL(ver), x));
10     if (op == 3) cout << pre(ALL(ver), x) - ver.begin() + 1 << endl;
11     if (op == 4) cout << ver[x - 1] << endl;
12     if (op == 5) cout << ver[pre(ALL(ver), x) - ver.begin() - 1] << endl;
13     if (op == 6) cout << ver[suf(ALL(ver), x) - ver.begin()] << endl;
14 }

```

8.17 线性基（高斯消元法）

设向量长度为 N （一般取 63），总数为 M ，时间复杂度为 $\mathcal{O}(NM)$ 。

```

1  struct LB { // Linear Basis
2      using i64 = long long;
3      const int BASE = 63;
4      vector<i64> d, p;
5      int cnt, flag;
6
7      LB() {
8          d.resize(BASE + 1);
9          p.resize(BASE + 1);
10         cnt = flag = 0;
11     }
12     bool insert(i64 val) {
13         for (int i = BASE - 1; i >= 0; i--) {
14             if (val & (1ll << i)) {
15                 if (!d[i]) {
16                     d[i] = val;
17                     return true;
18                 }
19                 val ^= d[i];
20             }
21         }
22         flag = 1; //可以异或出0
23         return false;
24     }
25     bool check(i64 val) { // 判断 val 是否能被异或得到
26         for (int i = BASE - 1; i >= 0; i--) {
27             if (val & (1ll << i)) {
28                 if (!d[i]) {
29                     return false;
30                 }
31                 val ^= d[i];
32             }
33         }
34         return true;
35     }
36     i64 ask_max() {
37         i64 res = 0;
38         for (int i = BASE - 1; i >= 0; i--) {
39             if ((res ^ d[i]) > res) res ^= d[i];
40         }

```

9 动态规划

9.1 01背包

有 n 件物品和一个容量为 W 的背包，第 i 件物品的体积为 $w[i]$ ，价值为 $v[i]$ ，求解将哪些物品装入背包中使总价值最大。

思路：

当放入一个体积为 $w[i]$ 的物品后，价值增加了 $v[i]$ ，于是我们可以构建一个二维的 $dp[i][j]$ 数组，装入第 i 件物品时，背包容量为 j 能实现的 **最大价值**，可以得到 **转移方程**
 $dp[i][j] = \max(dp[i-1][j], dp[i-1][j-w[i]] + v[i])$ 。

```
1 for (int i = 1; i <= n; i++)
2     for (int j = 0; j <= W; j++){
3         dp[i][j] = dp[i-1][j];
4         if (j >= w[i])
5             dp[i][j] = max(dp[i][j], dp[i-1][j-w[i]] + v[i]);
6     }
```

我们可以发现，第 i 个物品的状态是由第 $i-1$ 个物品转移过来的，每次的 j 转移过来后，第 $i-1$ 个方程的 j 已经没用了，于是我们想到可以把二维方程压缩成 **一维** 的，用以 **优化空间复杂度**。

```
1 for (int i = 1; i <= n; i++) //当前装第 i 件物品
2     for (int j = W; j >= w[i]; j--) //背包容量为 j
3         dp[j] = max(dp[j], dp[j-w[i]] + v[i]); //判断背包容量为 j 的情况下能是实现总
           价值最大是多少
```

9.2 完全背包

有 n 件物品和一个容量为 W 的背包，第 i 件物品的体积为 $w[i]$ ，价值为 $v[i]$ ，每件物品有**无限个**，求解将哪些物品装入背包中使总价值最大。

思路：

思路 and 01背包差不多，但是每一件物品有**无限个**，其实就是从每**种**物品中取 $0, 1, 2, \dots$ 件物品加入背包中

```
1 for (int i = 1; i <= n; i++)
2     for (int j = 0; j <= W; j++)
3         for (int k = 0; k * w[i] <= j; k++) //选取几个物品
4             dp[i][j] = max(dp[i][j], dp[i-1][j-k*w[i]] + k*v[i]);
```

实际上，我们可以发现，取 k 件物品可以从取 $k-1$ 件转移过来，那么我们就可以将 k 的循环优化掉

```
1 for (int i = 1; i <= n; i++)
2     for (int j = 0; j <= W; j++){
3         dp[i][j] = dp[i-1][j];
4         if (j >= w[i])
5             dp[i][j] = max(dp[i][j], dp[i][j-w[i]] + v[i]);
6     }
```

和 01 背包 类似地压缩成一维

```

1 for (int i = 1; i <= n; i++)
2     for (int j = w[i]; j <= W; j++)
3         dp[j] = max(dp[j], dp[j - w[i]] + v[i]);

```

9.3 多重背包

有 n 种物品和一个容量为 W 的背包，第 i 种物品的体积为 $w[i]$ ，价值为 $v[i]$ ，数量为 $s[i]$ ，求解将哪些物品装入背包中使总价值最大。

思路：

对于每一种物品，都有 $s[i]$ 种取法，我们可以将其转化为**01背包**问题

```

1 for (int i = 1; i <= n; i++){
2     for (int j = W; j >= 0; j--){
3         for (int k = 0; k <= s[i]; k++){
4             if (j - k * w[i] < 0) break;
5             dp[j] = max(dp[j], dp[j - k * w[i]] + k * v[i]);
6         }
7     }
8 }

```

上述方法的时间复杂度为 $O(n * m * s)$ 。

```

1 for (int i = 1; i <= n; i++){
2     scanf("%lld%lld%lld", &x, &y, &s); //x 为体积， y 为价值， s 为数量
3     t = 1;
4     while (s >= t){
5         w[++num] = x * t;
6         v[num] = y * t;
7         s -= t;
8         t *= 2;
9     }
10    w[++num] = x * s;
11    v[num] = y * s;
12 }
13 for (int i = 1; i <= num; i++)
14     for (int j = W; j >= w[i]; j--)
15         dp[j] = max(dp[j], dp[j - w[i]] + v[i]);

```

尽管采用了**二进制优化**，时间复杂度还是太高，采用**单调队列优化**，将时间复杂度优化至 $O(n * m)$

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 const int N = 2e5 + 10;
4 int n, W, w, v, s, f[N], g[N], q[N];
5 int main(){
6     ios::sync_with_stdio(false);cin.tie(0);
7     cin >> n >> W;
8     for (int i = 0; i < n; i++){
9         memcpy (g, f, sizeof f);
10        cin >> w >> v >> s;
11        for (int j = 0; j < w; j++){
12            int head = 0, tail = -1;
13            for (int k = j; k <= W; k += w){
14                if (head <= tail && k - s * w > q[head]) head++; //保证队列长度 <= s
15                while (head <= tail && g[q[tail]] - (q[tail] - j) / w * v <= g[k] - (k - j) / w * v) tail--; //保证队列单调递减
16                q[++tail] = k;
17                f[k] = g[q[head]] + (k - q[head]) / w * v;
18            }
19        }
20    }
21 }

```

```

18     }
19     }
20 }
21 cout << f[W] << "\n";
22 return 0;
23 }

```

9.4 混合背包

放入背包的物品可能只有 **1** 件（01背包），也可能有**无限件**（完全背包），也可能只有**可数的几件**（多重背包）。

思路：

分类讨论即可，哪一类就用哪种方法去 dp 。

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  int n, W, w, v, s;
4  int main(){
5      cin >> n >> W;
6      vector <int> f(W + 1);
7      for (int i = 0; i < n; i ++ ){
8          cin >> w >> v >> s;
9          if (s == -1){
10             for (int j = W; j >= w; j -- )
11                 f[j] = max(f[j], f[j - w] + v);
12         }
13         else if (s == 0){
14             for (int j = w; j <= W; j ++ )
15                 f[j] = max(f[j], f[j - w] + v);
16         }
17         else {
18             int t = 1, cnt = 0;
19             vector <int> x(s + 1), y(s + 1);
20             while (s >= t){
21                 x[++cnt] = w * t;
22                 y[cnt] = v * t;
23                 s -= t;
24                 t *= 2;
25             }
26             x[++cnt] = w * s;
27             y[cnt] = v * s;
28             for (int i = 1; i <= cnt; i ++ )
29                 for (int j = W; j >= x[i]; j -- )
30                     f[j] = max(f[j], f[j - x[i]] + y[i]);
31         }
32     }
33     cout << f[W] << "\n";
34     return 0;
35 }

```

9.5 二维费用的背包

有 n 件物品和一个容量为 W 的背包，背包能承受的最大重量为 M ，每件物品只能用一次，第 i 件物品的体积是 $w[i]$ ，重量为 $m[i]$ ，价值为 $v[i]$ ，求解将哪些物品放入背包中使总体积不超过背包容量，总重量不超过背包最大容量，且总价值最大。

思路：

背包的限制条件由一个变成两个，那么我们的循环再多一维即可。

```

1 for (int i = 1; i <= n; i++)
2     for (int j = W; j >= w; j--) //容量限制
3         for (int k = M; k >= m; k--) //重量限制
4             dp[j][k] = max(dp[j][k], dp[j - w][k - m] + v);

```

9.6 分组背包

有 n 组物品，一个容量为 W 的背包，每组物品有若干，同一组的物品最多选一个，第 i 组第 j 件物品的体积为 $w[i][j]$ ，价值为 $v[i][j]$ ，求解将哪些物品装入背包，可使物品总体积不超过背包容量，且使总价值最大。

思路：

考虑每组中的某件物品选不选，可以选的话，去下一组选下一个，否则在这组继续寻找可以选的物品，当这组遍历完后，去下一组寻找。

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 const int N = 110;
4 int n, W, s[N], w[N][N], v[N][N], dp[N];
5 int main(){
6     cin >> n >> W;
7     for (int i = 1; i <= n; i++){
8         scanf("%d", &s[i]);
9         for (int j = 1; j <= s[i]; j++)
10             scanf("%d %d", &w[i][j], &v[i][j]);
11     }
12     for (int i = 1; i <= n; i++)
13         for (int j = W; j >= 0; j--)
14             for (int k = 1; k <= s[i]; k++)
15                 if (j - w[i][k] >= 0)
16                     dp[j] = max(dp[j], dp[j - w[i][k]] + v[i][k]);
17     cout << dp[W] << "\n";
18     return 0;
19 }

```

9.7 有依赖的背包

有 n 个物品和一个容量为 W 的背包，物品之间有依赖关系，且之间的依赖关系组成一颗 **树** 的形状，如果选择一个物品，则必须选择它的 **父节点**，第 i 件物品的体积是 $w[i]$ ，价值为 $v[i]$ ，依赖的父节点的编号为 $p[i]$ ，若 $p[i]$ 等于 -1，则为 **根节点**。求将哪些物品装入背包中，使总体积不超过总容量，且总价值最大。

思路：

定义 $f[i][j]$ 为以第 i 个节点为根，容量为 j 的背包的最大价值。那么结果就是 $f[root][W]$ ，为了知道根节点的最大价值，得通过其子节点来更新。所以采用递归的方式。对于每一个点，先将这个节点装入背包，然后找到剩余容量可以实现的最大价值，最后更新父节点的最大价值即可。

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 const int N = 110;
4 int n, W, w[N], v[N], p, f[N][N], root;
5 vector<int> g[N];
6 void dfs(int u){
7     for (int i = w[u]; i <= W; i++)
8         f[u][i] = v[u];
9     for (auto v : g[u]){

```

```

10     dfs(v);
11     for (int j = W; j >= w[u]; j -- )
12         for (int k = 0; k <= j - w[u]; k ++ )
13             f[u][j] = max(f[u][j], f[u][j - k] + f[v][k]);
14     }
15 }
16 int main(){
17     cin >> n >> W;
18     for (int i = 1; i <= n; i ++ ){
19         cin >> w[i] >> v[i] >> p;
20         if (p == -1) root = i;
21         else g[p].push_back(i);
22     }
23     dfs(root);
24     cout << f[root][W] << "\n";
25     return 0;
26 }

```

9.8 背包问题求方案数

有 n 件物品和一个容量为 W 的背包，每件物品只能用一次，第 i 件物品的重量为 $w[i]$ ，价值为 $v[i]$ ，求解将哪些物品放入背包使总重量不超过背包容量，且总价值最大，输出 **最优选法的方案数**，答案可能很大，输出答案模 $10^9 + 7$ 的结果。

思路：

开一个储存方案数的数组 cnt ， $cnt[i]$ 表示容量为 i 时的 **方案数**，先将 cnt 的每一个值都初始化为 1，因为 **不装任何东西就是一种方案**，如果装入这件物品使总的价值 **更大**，那么装入后的方案数 **等于** 装之前的方案数，如果装入后总价值 **相等**，那么方案数就是 **二者之和**

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  #define LL long long
4  const int mod = 1e9 + 7, N = 1010;
5  LL n, W, cnt[N], f[N], w, v;
6  int main(){
7      cin >> n >> W;
8      for (int i = 0; i <= W; i ++ )
9          cnt[i] = 1;
10     for (int i = 0; i < n; i ++ ){
11         cin >> w >> v;
12         for (int j = W; j >= w; j -- )
13             if (f[j] < f[j - w] + v){
14                 f[j] = f[j - w] + v;
15                 cnt[j] = cnt[j - w];
16             }
17             else if (f[j] == f[j - w] + v){
18                 cnt[j] = (cnt[j] + cnt[j - w]) % mod;
19             }
20     }
21     cout << cnt[W] << "\n";
22     return 0;
23 }

```

9.9 背包问题求具体方案

```

1  signed main() {
2      int Task = 1;
3      for (cin >> Task; Task; Task--) {
4          int n, m;

```



```

5      cin >> n >> m;
6
7      vector<int> w(n + 1), v(n + 1);
8      vector<vector<int>> dp(n + 2, vector<int>(m + 2));
9      for (int i = 1; i <= n; i++) {
10         cin >> w[i] >> v[i];
11     }
12
13     for (int i = n; i >= 1; i--) {
14         for (int j = 0; j <= m; j++) {
15             dp[i][j] = dp[i + 1][j];
16             if (j >= w[i]) {
17                 dp[i][j] = max(dp[i][j], dp[i + 1][j - w[i]] + v[i]);
18             }
19         }
20     }
21
22     vector<int> ans;
23     for (int i = 1; i <= n; i++) {
24         if (m - w[i] >= 0 && dp[i][m] == dp[i + 1][m - w[i]] + v[i]) {
25             ans.push_back(i);
26             // cout << i << " ";
27             m -= w[i];
28         }
29     }
30     cout << ans.size() << "\n";
31     for (auto i : ans) {
32         cout << i << " ";
33     }
34     cout << "\n";
35 }
36 }

```

9.10 数位 DP

```

1  /* pos 表示当前枚举到第几位
2  sum 表示 d 出现的次数
3  limit 为 1 表示枚举的数字有限制
4  zero 为 1 表示有前导 0
5  d 表示要计算出出现次数的数 */
6  const int N = 15;
7  LL dp[N][N];
8  int num[N];
9  LL dfs(int pos, LL sum, int limit, int zero, int d) {
10     if (pos == 0) return sum;
11     if (!limit && !zero && dp[pos][sum] != -1) return dp[pos][sum];
12     LL ans = 0;
13     int up = (limit ? num[pos] : 9);
14     for (int i = 0; i <= up; i++) {
15         ans += dfs(pos - 1, sum + ((!zero || i) && (i == d)), limit && (i ==
num[pos]),
16                 zero && (i == 0), d);
17     }
18     if (!limit && !zero) dp[pos][sum] = ans;
19     return ans;
20 }
21 LL solve(LL x, int d) {
22     memset(dp, -1, sizeof dp);
23     int len = 0;
24     while (x) {
25         num[++len] = x % 10;
26         x /= 10;

```

```

27     }
28     return dfs(len, 0, 1, 1, d);
29 }

```

9.11 状压 DP

题意：在 $n * n$ 的棋盘里面放 k 个国王，使他们互不攻击，共有多少种摆放方案。国王能攻击到它上下左右，以及左上左下右上右下八个方向上附近的各一个格子，共8个格子。

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  #define LL long long
4  const int N = 15, M = 150, K = 1500;
5  LL n, k;
6  LL cnt[K];    //每个状态的二进制中 1 的数量
7  LL tot;       //合法状态的数量
8  LL st[K];     //合法的状态
9  LL dp[N][M][K]; //第 i 行，放置了 j 个国王，状态为 k 的方案数
10 int main(){
11     ios::sync_with_stdio(false);cin.tie(0);
12     cin >> n >> k;
13     for (int s = 0; s < (1 << n); s ++ ){ //找出合法状态
14         LL sum = 0, t = s;
15         while(t){ //计算 1 的数量
16             sum += (t & 1);
17             t >>= 1;
18         }
19         cnt[s] = sum;
20         if ( ((s << 1) | (s >> 1)) & s) == 0 ){ //判断合法性
21             st[ ++ tot] = s;
22         }
23     }
24     dp[0][0][0] = 1;
25     for (int i = 1; i <= n + 1; i ++ ){
26         for (int j1 = 1; j1 <= tot; j1 ++ ){ //当前的状态
27             LL s1 = st[j1];
28             for (int j2 = 1; j2 <= tot; j2 ++ ){ //上一行的状态
29                 LL s2 = st[j2];
30                 if ( ((s2 | (s2 << 1) | (s2 >> 1)) & s1) == 0 ){
31                     for (int j = 0; j <= k; j ++ ){
32                         if (j - cnt[s1] >= 0)
33                             dp[i][j][s1] += dp[i - 1][j - cnt[s1]][s2];
34                     }
35                 }
36             }
37         }
38     }
39     cout << dp[n + 1][k][0] << "\n";
40     return 0;
41 }

```

9.12 常用例题

题意：在一篇文章（包含大小写英文字母、数字、和空白字符（制表/空格/回车））中寻找 helloworld（任意一个字母的大小写都行）的子序列出现了多少次，输出结果对 $10^9 + 7$ 的余数。

字符串 DP，构建一个二维 DP 数组， $dp[i][j]$ 的 i 表示文章中的第几个字符， j 表示寻找的字符串的第几个字符，当字符串中的字符和文章中的字符相同时，即找到符合条件的字符， $dp[i][j] = dp[i - 1][j] + dp[i - 1][j - 1]$ ，因为字符串中的每个字符不会对后面的结果产生影响，所以 DP 方程可以优化成一维的，由于字符串中有重复的字符，所以比较时应该从后往前。

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  #define LL long long
4  const int mod = 1e9 + 7;
5  char c, s[20] = "!helloworld";
6  LL dp[20];
7  int main(){
8      dp[0] = 1;
9      while ((c = getchar()) != EOF)
10         for (int i = 10; i >= 1; i--)
11             if (c == s[i] || c == s[i] - 32)
12                 dp[i] = (dp[i] + dp[i - 1]) % mod;
13         cout << dp[10] << "\n";
14         return 0;
15     }

```

题意：（最长括号匹配）给一个只包含‘(’，‘)’，‘[’，‘]’的非空字符串，“()”和“[]”是匹配的，寻找字符串中最长的括号匹配的子串，若有两串长度相同，输出靠前的一串。

设给定的字符串为 s ，可以定义数组 $dp[i]$ ， $dp[i]$ 表示以 $s[i]$ 结尾的字符串里最长的括号匹配的字符。显然，从 $i - dp[i] + 1$ 到 i 的字符串是括号匹配的，当找到一个字符是‘)’或‘]’时，再去判断第 $i - 1 - dp[i - 1]$ 的字符和第 i 位的字符是否匹配，如果是，那么 $dp[i] = dp[i - 1] + 2 + dp[i - 2 - dp[i - 1]]$

。

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  const int maxn = 1e6 + 10;
4  string s;
5  int len, dp[maxn], ans, id;
6  int main(){
7      cin >> s;
8      len = s.length();
9      for (int i = 1; i < len; i++){
10         if ((s[i] == ')') && s[i - 1 - dp[i - 1]] == '(') || (s[i] == ']') && s[i - 1 - dp[i - 1]] == '['){
11             dp[i] = dp[i - 1] + 2 + dp[i - 2 - dp[i - 1]];
12             if (dp[i] > ans) {
13                 ans = dp[i]; //记录长度
14                 id = i; //记录位置
15             }
16         }
17     }
18     for (int i = id - ans + 1; i <= id; i++)
19         cout << s[i];
20     cout << "\n";
21     return 0;
22 }

```

题意：去掉区间内包含“4”和“62”的数字，输出剩余的数字个数

```

1  int T,n,m,len,a[20]; //a数组用于判断每一位能取到的最大值
2  ll l,r,dp[20][15];
3  ll dfs(int pos,int pre,int limit){ //记搜
4      //pos找到的位置，pre前一位数
5      //limit判断是否有最高位限制
6      if(pos>len) return 1; //剪枝
7      if(dp[pos][pre]!=-1 && !limit) return dp[pos][pre]; //记录当前值
8      ll ret=0; //暂时记录当前方案数

```

```

9      int res=limit?a[len-pos+1]:9;//res当前位能取到的最大值
10     for(int i=0;i<=res;i++)
11         if(!(i==4 || (pre==6 && i==2)))
12             ret+=dfs(pos+1,i,i==res&&limit);
13     if(!limit) dp[pos][pre]=ret;//当前状态方案数记录
14     return ret;
15 }
16 11 part(11 x){//把数按位拆分
17     len=0;
18     while(x) a[++len]=x%10,x/=10;
19     memset(dp,-1,sizeof dp);//初始化-1 (因为有可能某些情况下的方案数是0)
20     return dfs(1,0,1);//进入记搜
21 }
22 int main(){
23     cin>>n;
24     while(n--){
25         cin>>l>>r;
26         if(l==0 && r==0)break;
27         if(l) printf("%lld\n",part(r)-part(l-1));//[1,r](l!=0)
28         else printf("%lld\n",part(r)-part(1));//从0开始要特判
29     }
30 }

```

/END/

12 STL

12.1 库函数

12.1.1 pb_ds 库

其中 `gp_hash_table` 使用的最多，其等价于 `unordered_map`，内部是无序的。

```
1 #include <bits/extc++.h>
2 #include <ext/pb_ds/assoc_container.hpp>
3 template<class S, class T> using omap = __gnu_pbds::gp_hash_table<S, T, myhash>;
```

12.1.2 查找后继 lower_bound、upper_bound

`lower` 表示 $\geq$ ，`upper` 表示 $>$ 。使用前记得**先进行排序**。

```
1 //返回a数组[start,end)区间中第一个>=x的地址【地址!!!】
2 cout << lower_bound(a + start, a + end, x);
3
4 cout << lower_bound(a, a + n, x) - a; //在a数组中查找第一个>=x的元素下标
5 upper_bound(a, a + n, k) - lower_bound(a, a + n, k) //查找k在a中出现了几次
```

12.1.3 数组打乱 shuffle

```
1 mt19937 rnd(chrono::steady_clock::now().time_since_epoch().count());
2 shuffle(ver.begin(), ver.end(), rnd);
```

12.1.4 二分搜索 binary_search

用于查找某一元素是否在容器中，相当于 `find` 函数。在使用前需要**先进行排序**。

```
1 //在a数组[start,end)区间中查找x是否存在，返回bool型
2 cout << binary_search(a + start, a + end, x);
```

12.1.5 批量递增赋值函数 iota

对容器递增初始化。

```
1 //将a数组[start,end)区间复制成“x, x+1, x+2, ...”
2 iota(a + start, a + end, x);
```

12.1.6 数组去重函数 unique

在使用前需要**先进行排序**。

其作用是，对于区间 `[开始位置, 结束位置)`，不停的把后面不重复的元素移到前面来，也可以说是用**不重复的元素占领重复元素的位置**。并且返回去重后容器中不重复序列的最后一个元素的下一个元素。所以在进行操作后，数组、容器的大小并没有发生改变。

```
1 //将a数组[start,end)区间去重，返回迭代器
2 unique(a + start, a + end);
3
4 //与erase函数结合，达到去重+删除的目的
5 a.erase(unique(ALL(a)), a.end());
```

12.1.7 bit 库与位运算函数 `__builtin_`

```

1  __builtin_popcount(x) // 返回x二进制下含1的数量，例如x=15=(1111)时答案为4
2
3  __builtin_ffs(x) // 返回x右数第一个1的位置(1-idx)，1(1) 返回 1，8(1000) 返回 4，
   26(11010) 返回 2
4
5  __builtin_ctz(x) // 返回x二进制下后导0的个数，1(1) 返回 0，8(1000) 返回 3
6
7  bit_width(x) // 返回x二进制下的位数，9(1001) 返回 4，26(11010) 返回 5

```

注：以上函数的 `long long` 版本只需要在函数后面加上 `ll` 即可（例如 `__builtin_popcountll(x)`），`unsigned long long` 加上 `ull`。

12.1.8 数字转字符串函数

`itoa` 虽然能将整数转换成任意进制的字符串，但是其不是标准的C函数，且为Windows独有，且不支持 `long long`，建议手写。

```

1  // to_string函数会直接将你的各种类型的数字转换为字符串。
2  // string to_string(T val);
3  double val = 12.12;
4  cout << to_string(val);

```

```

1  // 【不建议使用】itoa允许你将整数转换成任意进制的字符串，参数为待转换整数、目标字符数组、进制。
2  // char* itoa(int value, char* string, int radix);
3  char ans[10] = {};
4  itoa(12, ans, 2);
5  cout << ans << endl; /*1100*/
6
7  // 长整型函数名ltoa，最高支持到int型上限2^31。ultoa同理。

```

12.1.9 字符串转数字

```

1  // stoi直接使用
2  cout << stoi("12") << endl;
3
4  // 【不建议使用】stoi转换进制，参数为待转换字符串、起始位置、进制。
5  // int stoi(string value, int st, int radix);
6  cout << stoi("1010", 0, 2) << endl; /*10*/
7  cout << stoi("c", 0, 16) << endl; /*12*/
8  cout << stoi("0xf3f3f3f", 0, 0) << endl; /*1061109567*/
9
10 // 长整型函数名stoll，最高支持到long long型上限2^63。stoull、stod、stold同理。

```

```

1  // atoi直接使用，空字符返回0，允许正负符号，数字字符前有其他字符返回0，数字字符前有空白字符自动去除
2  cout << atoi("12") << endl;
3  cout << atoi(" 12") << endl; /*12*/
4  cout << atoi("-12abc") << endl; /*-12*/
5  cout << atoi("abc12") << endl; /*0*/
6
7  // 长整型函数名atoll，最高支持到long long型上限2^63。

```

12.1.10 全排列算法 next_permutation、prev_permutation

在提及这个函数时，我们先需要补充几点字典序相关的知识。

对于三个字符所组成的序列 {a,b,c}，其按照字典序的6种排列分别为：

{abc} , {acb} , {bac} , {bca} , {cab} , {cba}

其排序原理是：先固定 a (序列内最小元素)，再对之后的元素排列。而 $b < c$ ，所以 $abc < acb$ 。同理，先固定 b (序列内次小元素)，再对之后的元素排列。即可得出以上序列。

next_permutation 算法，即是按照字典序顺序输出的全排列；相对应的，prev_permutation 则是按照逆字典序顺序输出的全排列。可以是数字，亦可以是其他类型元素。其直接在序列上进行更新，故直接输出序列即可。

```
1 int n;
2 cin >> n;
3 vector<int> a(n);
4 // iota(a.begin(), a.end(), 1);
5 for (auto &it : a) cin >> it;
6 sort(a.begin(), a.end());
7
8 do {
9     for (auto it : a) cout << it << " ";
10    cout << endl;
11 } while (next_permutation(a.begin(), a.end()));
```

12.1.11 字符串转换为数值函数 stoi

可以快捷的将一串字符串转换为指定进制的数字。

使用方法

- stoi(字符串, 0, x进制)：将一串 x 进制的字符串转换为 int 型数字。

```
void Solve() {
    cout << stoi("1010", 0, 2) << endl;
    cout << stoi("c", 0, 16) << endl;
    cout << stoi("0x3f3f3f3f", 0, 0) << endl;
    cout << stoi("10", 0, 8) << endl;
    cout << stoll("aaaaaaaaaa", 0, 16) << endl;
}
```

C:\Users\26099\Desktop\万能头文件.exe

```
10
12
1061109567
8
11728124029610
```

- stoll(字符串, 0, x进制)：将一串 x 进制的字符串转换为 long long 型数字。
- stoull, stod, stold 同理。

12.1.12 数值转换为字符串函数 to_string

允许将各种数值类型转换为字符串类型。

```
1 //将数值num转换为字符串s
2 string s = to_string(num);
```

12.1.13 判断非递减 is_sorted

```
1 //a数组[start,end)区间是否是非递减的，返回bool型
2 cout << is_sorted(a + start, a + end);
```

12.1.14 累加 accumulate

```
1 //将a数组[start,end)区间的元素进行累加，并输出累加和+x的值
2 cout << accumulate(a + start, a + end, x);
```

12.1.15 迭代器 iterator

```
1 //构建一个UUU容器的正向迭代器，名字叫it
2 UUU::iterator it;
3
4 vector<int>::iterator it; //创建一个正向迭代器，++ 操作时指向下一个
5 vector<int>::reverse_iterator it; //创建一个反向迭代器，++ 操作时指向上一个
```

12.1.16 其他函数

`exp2(x)` : 返回 2^x

`log2(x)` : 返回 $\log_2(x)$

`gcd(x, y) / lcm(x, y)` : 以 $\log$ 的复杂度返回 $\gcd(|x|, |y|)$ 与 $\text{lcm}(|x|, |y|)$ ，且返回值符号也为正数。

12.2 容器与成员函数

12.2.1 元组 tuple

```
1 //获取obj对象中的第index个元素—get<index>(obj)
2 //需要注意的是这里的index只能手动输入，使用for循环这样的自动输入是不可以的
3 tuple<string, int, int> Student = {"Wida", 23, 45000};
4 cout << get<0>(Student) << endl; //获取Student对象中的第一个元素，这里的输出结果应
   为"Wida"
```

12.2.2 数组 array

```
1 array<int, 3> x; // 建立一个包含三个元素的数组x
2
3 [] // 跟正常数组一样，可以使用随机访问
4 cout << x[0]; // 获取数组重的第一个元素
```


12.2.3 变长数组 vector

```

1  resize(n) // 重设容器大小，但是不改变已有元素的值
2  assign(n, 0) // 重设容器大小为n，且替换容器内的内容为0
3
4  // 尽量不要使用[]的形式声明多维变长数组，而是使用嵌套的方式替代
5  vector<int> ver[n + 1]; // 不好的声明方式
6  vector<vector<int>> ver(n + 1);
7
8  // 嵌套时只需要在最后一个注明变量类型
9  vector dis(n + 1, vector<int>(m + 1));
10 vector dis(m + 1, vector(n + 1, vector<int>(n + 1)));

```

12.2.4 栈 stack

栈顶入，栈顶出。先进后出。

```

1  //没有clear函数
2  size() / empty()
3  push(x) //向栈顶插入x
4  top() //获取栈顶元素
5  pop() //弹出栈顶元素

```

12.2.5 队列 queue

队尾进，队头出。先进先出。

```

1  //没有clear函数
2  size() / empty()
3  push(x) //向队尾插入x
4  front() / back() //获取队头、队尾元素
5  pop() //弹出队头元素

```

```

1  //没有clear函数，但是可以用重新构造替代
2  queue<int> q;
3  q = queue<int>();

```

12.2.6 双向队列 deque

```

1  size() / empty() / clear()
2  push_front(x) / push_back(x)
3  pop_front(x) / pop_back(x)
4  front() / back()
5  begin() / end()
6  []

```

12.2.7 优先队列 priority_queue

默认升序（大根堆），自定义排序需要重载 `<`。

```

1  //没有clear函数
2  priority_queue<int, vector<int>, greater<int> > p; //重定义为降序（小根堆）
3  push(x); //向栈顶插入x
4  top(); //获取栈顶元素
5  pop(); //弹出栈顶元素

```

```

1 //重载运算符【注意，符号相反！！】
2 struct Node {
3     int x; string s;
4     friend bool operator < (const Node &a, const Node &b) {
5         if (a.x != b.x) return a.x > b.x;
6         return a.s > b.s;
7     }
8 };

```

12.2.8 字符串 string

```

1 size() / empty() / clear()

```

```

1 //从字符串s的S[start]开始，取出长度为len的子串—S.substr(start, len)
2 //len省略时默认取到结尾，超过字符串长度时也默认取到结尾
3 cout << S.substr(1, 12);
4
5 find(x) / rfind(x); //顺序、逆序查找x，返回下标，没找到时返回一个极大值【! 建议与 size() 比
6 //较，而不要和 -1 比较，后者可能出错】
7 //注意，没有count函数

```

12.2.9 有序、多重有序集合 set、multiset

默认升序（大根堆），set 去重，multiset 不去重， $\mathcal{O}(\log N)$ 。

```

1 set<int, greater<>> > s; //重定义为降序（小根堆）
2 size() / empty() / clear()
3 begin() / end()
4 ++ / -- //返回前驱、后继
5
6 insert(x); //插入x
7 find(x) / rfind(x); //顺序、逆序查找x，返回迭代器【迭代器！！】，没找到时返回end()
8 count(x); //返回x的个数
9 lower_bound(x); //返回第一个>=x的迭代器【迭代器！！】
10 upper_bound(x); //返回第一个>x的迭代器【迭代器！！】

```

特殊函数 next 和 prev 详解：

```

1 auto it = s.find(x); // 建立一个迭代器
2 prev(it) / next(it); // 默认返回迭代器it的前/后一个迭代器
3 prev(it, 2) / next(it, 2); // 可选参数可以控制返回前/后任意个迭代器
4
5 /* 以下是一些应用 */
6 auto pre = prev(s.lower_bound(x)); // 返回第一个< x的迭代器
7 int ed = *prev(S.end(), 1); // 返回最后一个元素

```

erase(x); 有两种删除方式：

- 当x为某一元素时，删除**所有**这个数，复杂度为 $\mathcal{O}(num_x + \log N)$ ；
- 当x为迭代器时，删除这个迭代器。

```

1 //连续头部删除
2 set<int> S = {0, 9, 98, 1087, 894, 34, 756};
3 auto it = S.begin();
4 int len = S.size();
5 for (int i = 0; i < len; ++ i) {
6     if (*it >= 500) continue;
7     it = S.erase(it); //删除所有小于500的元素
8 }
9 //错误用法如下【千万不能这样用!!!】
10 //for (auto it : S) {
11 //    if (it >= 500) continue;
12 //    S.erase(it); //删除所有小于500的元素
13 //}

```

12.2.10 map、multimap

默认升序（大根堆），map 去重，multimap 不去重， $O(\log S)$ ，其中 S 为元素数量。

```

1 map<int, int, greater<>> mp; //重定义为降序（小根堆）
2 size() / empty() / clear()
3 begin() / end()
4 ++ / -- //返回前驱、后继
5
6 insert({x, y}); //插入二元组
7 [] //随机访问，multimap不支持
8 count(x); //返回x为下标的个数
9 lower_bound(x); //返回第一个下标>=x的迭代器
10 upper_bound(x); //返回第一个下标>x的迭代器

```

erase(x); 有两种删除方式：

- 当x为某一元素时，删除所有以这个元素为下标的二元组，复杂度为 $O(\text{num}_x + \log N)$ ；
- 当x为迭代器时，删除这个迭代器。

慎用随机访问！——当不确定某次查询是否存在于容器中时，不要直接使用下标查询，而是先使用 count() 或者 find() 方法检查key值，防止不必要的零值二元组被构造。

```

1 int q = 0;
2 if (mp.count(i)) q = mp[i];

```

慎用自带的 pair、tuple 作为key值类型！使用自定义结构体！

```

1 struct fff {
2     LL x, y;
3     friend bool operator < (const fff &a, const fff &b) {
4         if (a.x != b.x) return a.x < b.x;
5         return a.y < b.y;
6     }
7 };
8 map<fff, int> mp;

```

12.2.11 bitset

将数据转换为二进制，从高位到低位排序，以 0 为最低位。当位数相同时支持全部的位运算。

```

1 // 如果输入的是01字符串，可以直接使用">>"读入
2 bitset<10> s;

```

```

3  cin >> s;
4
5  //使用只含01的字符串构造—bitset<容器长度>B (字符串)
6  string S; cin >> S;
7  bitset<32> B (S);
8
9  //使用整数构造 (两种方式)
10 int x; cin >> x;
11 bitset<32> B1 (x);
12 bitset<32> B2 = x;
13
14 // 构造时,尖括号里的数字不能是变量
15 int x; cin >> x;
16 bitset<x> ans; // 错误构造
17
18 [] //随机访问
19 set(x) //将第x位置1, x省略时默认全部位置1
20 reset(x) //将第x位置0, x省略时默认全部位置0
21 flip(x) //将第x位取反, x省略时默认全部位取反
22 to_ulong() //重转换为ULL类型
23 to_string() //重转换为ULL类型
24 count() //返回1的个数
25 any() //判断是否至少有一个1
26 none() //判断是否全为0
27
28 _Find_fisrt() // 找到从低位到高位第一个1的位置
29 _Find_next(x) // 找到当前位置x的下一个1的位置, 复杂度 O(n/w + count)
30
31 bitset<23> B1("11101001"), B2("11101000");
32 cout << (B1 ^ B2) << "\n"; //按位异或
33 cout << (B1 | B2) << "\n"; //按位或
34 cout << (B1 & B2) << "\n"; //按位与
35 cout << (B1 == B2) << "\n"; //比较是否相等
36 cout << B1 << " " << B2 << "\n"; //你可以直接使用cout输出

```

12.2.12 哈希系列 unordered

通常指代 unordered_map、unordered_set、unordered_multimap、unordered_multiset，与原版相比不进行排序。

如果将不支持哈希的类型作为 `key` 值代入，编译器就无法正常运行，这时需要我们为其手写哈希函数。而我们写的这个哈希函数的正确性其实并不是特别重要（但是不可以没有），当发生冲突时编译器会调用 `key` 的 `operator ==` 函数进行进一步判断。[参考](#)

12.2.13 对 pair、tuple 定义哈希

```

1  struct hash_pair {
2      template <class T1, class T2>
3      size_t operator()(const pair<T1, T2> &p) const {
4          return hash<T1>()(p.fi) ^ hash<T2>()(p.se);
5      }
6  };
7  unordered_set<pair<int, int>, int, hash_pair> S;
8  unordered_map<tuple<int, int, int>, int, hash_pair> M;

```

12.2.14 对结构体定义哈希

需要两个条件，一个是在结构体中重载等于号（区别于非哈希容器需要重载小于号，如上所述，当冲突时编译器需要根据重载的等于号判断），第二是写一个哈希函数。注意 `hash<>()` 的尖括号中的类型匹配。

```
1 struct fff {
2     string x, y;
3     int z;
4     friend bool operator == (const fff &a, const fff &b) {
5         return a.x == b.x || a.y == b.y || a.z == b.z;
6     }
7 };
8 struct hash_fff {
9     size_t operator()(const fff &p) const {
10         return hash<string>()(p.x) ^ hash<string>()(p.y) ^ hash<int>()(p.z);
11     }
12 };
13 unordered_map<fff, int, hash_fff> mp;
```

12.2.15 对 vector 定义哈希

以下两个方法均可。注意 `hash<>()` 的尖括号中的类型匹配。

```
1 struct hash_vector {
2     size_t operator()(const vector<int> &p) const {
3         size_t seed = 0;
4         for (auto it : p) {
5             seed ^= hash<int>()(it);
6         }
7         return seed;
8     }
9 };
10 unordered_map<vector<int>, int, hash_vector> mp;
```

```
1 namespace std {
2     template<> struct hash<vector<int>> {
3         size_t operator()(const vector<int> &p) const {
4             size_t seed = 0;
5             for (int i : p) {
6                 seed ^= hash<int>()(i) + 0x9e3779b9 + (seed << 6) + (seed >> 2);
7             }
8             return seed;
9         }
10    };
11 }
12 unordered_set<vector<int>> S;
```

12.3 程序标准化

12.3.1 使用 Lambda 函数

- `function` 统一写法

需要注意的是，虽然 `function` 定义时已经声明了返回值类型了，但是有的时候会出错（例如，声明返回 `long long` 但是返回 `int`，原因没去了解），所以推荐在后面使用 `->` 再行声明一遍。

```

1 function<void(int, int)> clac = [&](int x, int y) -> void {
2 };
3 clac(1, 2);
4
5 function<bool(int)> dfs = [&](int x) -> bool {
6     return dfs(x + 1);
7 };
8 dfs(1);

```

- `auto` 非递归写法

不需要使用递归函数时，直接用 `auto` 替换 `function` 即可。

```

1 auto clac = [&](int x, int y) -> void {
2 };

```

- `auto` 递归写法

相较于 `function` 写法，需要额外引用一遍自身。

```

1 auto dfs = [&](auto self, int x) -> bool {
2     return self(self, x + 1);
3 };
4 dfs(dfs, 1);

```

12.3.2 使用构造函数

可以将一些必要的声明和预处理放在构造函数，在编译时，无论放置在程序的哪个位置，都会先于主函数进行。下方是我将输入流控制声明的过程。

```

1 int __FAST_IO__ = []() { // 函数名称可以随意修改
2     ios::sync_with_stdio(0), cin.tie(0);
3     cout.tie(0);
4     cout << fixed << setprecision(12);
5     freopen("out.txt", "r", stdin);
6     freopen("in.txt", "w", stdout);
7     return 0;
8 }();

```

/END/

14 杂类

14.1 统计区间不同数字的数量（离线查询）

核心在于使用 `pre` 数组滚动维护每一个数字出现的最后位置，配以树状数组统计数量。由于滚动维护具有后效性，所以需要离线操作，从前往后更新。时间复杂度 $\mathcal{O}(N \log N)$ ，常数瓶颈在于 `map`，用手造哈希或者离散化可以优化到理想区间；同时也有莫队做法，复杂度稍劣。[例题链接](#)。

```

1  signed main() {
2      int n;
3      cin >> n;
4      vector<int> in(n + 1);
5      for (int i = 1; i <= n; i++) {
6          cin >> in[i];
7      }
8
9      int q;
10     cin >> q;
11     vector<array<int, 3>> query;
12     for (int i = 0; i < q; i++) {
13         int l, r;
14         cin >> l >> r;
15         query.push_back({r, l, i});
16     }
17     sort(query.begin(), query.end());
18
19     vector<pair<int, int>> ans;
20     map<int, int> pre;
21     int st = 1;
22     BIT bit(n);
23     for (auto [r, l, id] : query) {
24         for (int i = st; i <= r; i++, st++) {
25             if (pre.count(in[i])) { // 消除此前操作的影响
26                 bit.add(pre[in[i]], -1);
27             }
28             bit.add(i, 1);
29             pre[in[i]] = i; // 更新操作
30         }
31         ans.push_back({id, bit.ask(r) - bit.ask(l - 1)});
32     }
33
34     sort(ans.begin(), ans.end());
35     for (auto [id, w] : ans) {
36         cout << w << endl;
37     }
38 }

```

14.2 选数（DFS 解）

从 N 个整数中任选 K 个整数相加。使用 DFS 求解。

```

1  int n, k; cin >> n >> k;
2  vector<int> in(n), now(n);
3  for (auto &it : in) { cin >> it; }
4  auto dfs = [&](auto self, int k, int bit, int idx) -> void {
5      for (int i = idx; i < n; i++) {
6          now[bit] = in[i];
7          if (bit < k - 1) { self(self, k, bit + 1, i + 1); }
8          if (bit == k - 1) {
9              int add = 0;

```

```

10         for (int j = 0; j < k; j++) {
11             add += now[j];
12         }
13         cout << add << endl;
14     }
15 }
16 };
17 dfs(dfs, k, 0, 0);

```

14.3 选数（位运算状压）

```

1  int n, k; cin >> n >> k;
2  vector<int> in(n);
3  for (auto &it : in) { cin >> it; }
4  int comb = (1 << k) - 1, U = 1 << n;
5  while (comb < U) {
6      int add = 0;
7      for (int i = 0; i < n; i++) {
8          if (1 << i & comb) {
9              add += in[i];
10         }
11     }
12     cout << add << "\n";
13
14     int x = comb & -comb;
15     int y = comb + x;
16     int z = comb & ~y;
17     comb = (z / x >> 1) | y;
18 }

```

14.4 网格路径计数

从 $(0, 0)$ 走到 (a, b) ，规定每次只能从 (x, y) 走到左下或者右下，方案数记为 $f(a, b)$ 。

- $f(a, b) = \binom{a}{\frac{a+b}{2}}$ ；
- 若路径和直线 $y = k, k \notin [0, b]$ 不能有交点，则方案数为 $f(a, b) - f(a, 2k - b)$ ；
- 若路径和两条直线 $y = k_1, y = k_2$ ($k_1 < 0 \leq b < k_2$) 不能有交点，方案数记为 $g(a, b, k_1, k_2)$ ，可以使用 $\mathcal{O}(N)$ 递归求解；
- 若路径必须碰到 $y = k_1$ 但是不能碰到 $y = k_2$ ，方案数记为 $h(a, b, k_1, k_2)$ ，可以使用 $\mathcal{O}(N)$ 递归求解（递归过程中两条直线距离会越来越大）。

从 $(0, 0)$ 走到 $(a, 0)$ ，规定每次只能走到左下或者右下，且必须有**恰好一次**传送（向下 b 单位），且不能走到 x 轴下方，方案数为 $\binom{a+1}{\frac{a-b}{2} + k + 1}$ 。

14.5 德州扑克

读入牌型，并且支持两副牌之间的大小比较。[代码参考](#)

```

1  struct card {
2      int suit, rank;
3      friend bool operator < (const card &a, const card &b) {
4          return a.rank < b.rank;
5      }
6      friend bool operator == (const card &a, const card &b) {
7          return a.rank == b.rank;
8      }

```



```

9      friend bool operator != (const card &a, const card &b) {
10          return a.rank != b.rank;
11      }
12      friend auto &operator>> (istream &it, card &C) {
13          string S, T; it >> S;
14          T = "__23456789TJQKA"; //点数
15          FOR (i, 0, T.sz - 1) {
16              if (T[i] == S[0]) C.rank = i;
17          }
18          T = "_SHCD"; //花色
19          FOR (i, 0, T.sz - 1) {
20              if (T[i] == S[1]) C.suit = i;
21          }
22          return it;
23      }
24  };
25  struct game {
26      int level;
27      vector<card> peo;
28      int a, b, c, d, e;
29      int u, v, w, x, y;
30      bool Rk10() { //Rk10: Royal Flush, 五张牌同花色, 且点数为AKQJT (14,13,12,11,10)
31          sort(ALL(peo));
32          reverse(ALL(peo));
33          a = peo[0].rank, b = peo[1].rank, c = peo[2].rank, d = peo[3].rank, e =
34          peo[4].rank;
35          u = peo[0].suit, v = peo[1].suit, w = peo[2].suit, x = peo[3].suit, y =
36          peo[4].suit;
37
38          if (u != v || v != w || w != x || x != y) return 0;
39          if (a == 14 && b == 13 && c == 12 && d == 11 && e == 10) return 1;
40          return 0;
41      }
42      bool Dif(vector<card> &peo) { //专门用于检查A2345这种顺子的情况 (这是最小的顺子)
43          a = peo[0].rank, b = peo[1].rank, c = peo[2].rank, d = peo[3].rank, e =
44          peo[4].rank;
45          u = peo[0].suit, v = peo[1].suit, w = peo[2].suit, x = peo[3].suit, y =
46          peo[4].suit;
47
48          if (a != 14 || b != 5 || c != 4 || d != 3 || e != 2) return 0;
49          vector<card> peo2 = {peo[1], peo[2], peo[3], peo[4], peo[0]}; //重新排序
50          peo = peo2;
51          return 1;
52      }
53      bool Rk9() { //Rk9: Straight Flush, 五张牌同花色, 且顺连【r1 > r2 > r3 > r4 >
54          r5】
55          sort(ALL(peo));
56          reverse(ALL(peo));
57          a = peo[0].rank, b = peo[1].rank, c = peo[2].rank, d = peo[3].rank, e =
58          peo[4].rank;
59          u = peo[0].suit, v = peo[1].suit, w = peo[2].suit, x = peo[3].suit, y =
60          peo[4].suit;
61
62          if (u != v || v != w || w != x || x != y) return 0;
63          if (Dif(peo)) return 1; //特判: A2345
64          if (a == b + 1 && b == c + 1 && c == d + 1 && d == e + 1) return 1;
65          return 0;
66      }
67      bool Rk8() { //Rk8: Four of a Kind, 四张牌点数一样【r1 = r2 = r3 = r4】
68          sort(ALL(peo));
69          a = peo[0].rank, b = peo[1].rank, c = peo[2].rank, d = peo[3].rank, e =
70          peo[4].rank;
71          u = peo[0].suit, v = peo[1].suit, w = peo[2].suit, x = peo[3].suit, y =
72          peo[4].suit;

```

```

64
65     if (a == b && b == c && c == d) return 1;
66     if (b == c && c == d && d == e) {
67         reverse(ALL(peo));
68         return 1;
69     }
70     return 0;
71 }
72 bool Rk7() { //Rk7: Fullhouse, 三张牌点数一样, 另外两张点数也一样【r1 = r2 = r3, r4
= r5】
73     sort(ALL(peo));
74     a = peo[0].rank, b = peo[1].rank, c = peo[2].rank, d = peo[3].rank, e =
75     peo[4].rank;
76     u = peo[0].suit, v = peo[1].suit, w = peo[2].suit, x = peo[3].suit, y =
77     peo[4].suit;
78     if (a == b && b == c && d == e) return 1;
79     if (a == b && c == d && d == e) {
80         reverse(ALL(peo));
81         return 1;
82     }
83     return 0;
84 }
85 bool Rk6() { //Rk6: Flush, 五张牌同花色【r1 > r2 > r3 > r4 > r5】
86     sort(ALL(peo));
87     reverse(ALL(peo));
88     a = peo[0].rank, b = peo[1].rank, c = peo[2].rank, d = peo[3].rank, e =
89     peo[4].rank;
90     u = peo[0].suit, v = peo[1].suit, w = peo[2].suit, x = peo[3].suit, y =
91     peo[4].suit;
92     if (u != v || v != w || w != x || x != y) return 0;
93     return 1;
94 }
95 bool Rk5() { //Rk5: Straight, 五张牌顺连【r1 > r2 > r3 > r4 > r5】
96     sort(ALL(peo));
97     reverse(ALL(peo));
98     a = peo[0].rank, b = peo[1].rank, c = peo[2].rank, d = peo[3].rank, e =
99     peo[4].rank;
100    u = peo[0].suit, v = peo[1].suit, w = peo[2].suit, x = peo[3].suit, y =
101    peo[4].suit;
102    if (Dif(peo)) return 1; //特判: A2345
103    if (a == b + 1 && b == c + 1 && c == d + 1 && d == e + 1) return 1;
104    return 0;
105 }
106 bool Rk4() { //Rk4: Three of a kind, 三张牌点数一样【r1 = r2 = r3, r4 > r5】
107     sort(ALL(peo));
108     reverse(ALL(peo));
109     a = peo[0].rank, b = peo[1].rank, c = peo[2].rank, d = peo[3].rank, e =
110     peo[4].rank;
111     u = peo[0].suit, v = peo[1].suit, w = peo[2].suit, x = peo[3].suit, y =
112     peo[4].suit;
113     if (a == b && b == c) return 1;
114     if (b == c && c == d) {
115         swap(peo[3], peo[0]);
116         return 1;
117     }
118     if (c == d && d == e) {
119         swap(peo[3], peo[0]);
120         swap(peo[4], peo[1]);
121         return 1;
122     }
123 }

```

```

119         return 0;
120     }
121     bool Rk3() { //Rk3: Two Pairs , 两张牌点数一样 , 另外有两张点数也一样 ( 两个对子 ) 【r1
= r2 > r3 = r4】
122         sort(ALL(peo));
123         reverse(ALL(peo));
124         a = peo[0].rank, b = peo[1].rank, c = peo[2].rank, d = peo[3].rank, e =
peo[4].rank;
125         u = peo[0].suit, v = peo[1].suit, w = peo[2].suit, x = peo[3].suit, y =
peo[4].suit;
126
127         if (a == b && c == d) return 1;
128         if (a == b && d == e) {
129             swap(peo[2], peo[4]);
130             return 1;
131         }
132         if (b == c && d == e) {
133             swap(peo[0], peo[2]);
134             swap(peo[2], peo[4]);
135             return 1;
136         }
137         return 0;
138     }
139     bool Rk2() { //Rk2: One Pairs , 两张牌点数一样 ( 一个对子 ) 【r1 = r2 , r3 > r4 > r5】
140         sort(ALL(peo));
141         reverse(ALL(peo));
142         a = peo[0].rank, b = peo[1].rank, c = peo[2].rank, d = peo[3].rank, e =
peo[4].rank;
143         u = peo[0].suit, v = peo[1].suit, w = peo[2].suit, x = peo[3].suit, y =
peo[4].suit;
144
145         vector<card> peo2;
146         if (a == b) return 1;
147         if (b == c) {
148             peo2 = {peo[1], peo[2], peo[0], peo[3], peo[4]};
149             peo = peo2;
150             return 1;
151         }
152         if (c == d) {
153             peo2 = {peo[2], peo[3], peo[0], peo[1], peo[4]};
154             peo = peo2;
155             return 1;
156         }
157         if (d == e) {
158             peo2 = {peo[3], peo[4], peo[0], peo[1], peo[2]};
159             peo = peo2;
160             return 1;
161         }
162         return 0;
163     }
164     bool Rk1() { //Rk1: high card
165         sort(ALL(peo));
166         reverse(ALL(peo));
167         return 1;
168     }
169     game (vector<card> New_peo) {
170         peo = New_peo;
171         if (Rk10()) { level = 10; return; }
172         if (Rk9()) { level = 9; return; }
173         if (Rk8()) { level = 8; return; }
174         if (Rk7()) { level = 7; return; }
175         if (Rk6()) { level = 6; return; }
176         if (Rk5()) { level = 5; return; }
177         if (Rk4()) { level = 4; return; }

```

```

178         if (Rk3()) { level = 3; return; }
179         if (Rk2()) { level = 2; return; }
180         if (Rk1()) { level = 1; return; }
181     }
182     friend bool operator < (const game &a, const game &b) {
183         if (a.level != b.level) return a.level < b.level;
184         FOR (i, 0, 4) if (a.peo[i] != b.peo[i]) return a.peo[i] < b.peo[i];
185         return 0;
186     }
187     friend bool operator == (const game &a, const game &b) {
188         if (a.level != b.level) return 0;
189         FOR (i, 0, 4) if (a.peo[i] != b.peo[i]) return 0;
190         return 1;
191     }
192 };
193 void debug(vector<card> peo) {
194     for (auto it : peo) cout << it.rank << " " << it.suit << " ";
195     cout << "\n\n";
196 }
197 int clac(vector<card> Ali, vector<card> Bob) {
198     game atype(Ali), btype(Bob);
199     if (atype < btype) return -1;
200     else if (atype == btype) return 0;
201     return 1;
202 }

```

14.6 N*M 数独字典序最小方案

规则：每个宫大小为 $2^N * 2^M$ ，大图一共由 $M * N$ 个宫组成（总大小即 $2^N 2^M * 2^N 2^M$ ），要求每行、每列、每宫都要出现 1 到 $2^N * 2^M$ 的全部数字。输出字典序最小方案。

下例为 2, 1 和 1, 2 时数独字典序最小的示意。

	0	1	2	3	4	5	6	7			0	1	2	3	4	5	6	7		
0	1	2	3	4	5	6	7	8			0	1	2	3	4	5	6	7	8	
1	3	4	1	2	7	8	5	6			1	5	6	7	8	1	2	3	4	
2	5	6	7	8	1	2	3	4			2	2	1	4	3	6	5	8	7	
3	7	8	5	6	3	4	1	2			3	6	5	8	7	2	1	4	3	
4	2	1	4	3	6	5	8	7			4	3	4	1	2	7	8	5	6	
5	4	3	2	1	8	7	6	5			5	7	8	5	6	3	4	1	2	
6	6	5	8	7	2	1	4	3			6	4	3	2	1	8	7	6	5	
7	8	7	6	5	4	3	2	1			7	8	7	6	5	4	3	2	1	

公式： (i, j) 格所填的内容为 $(i \bmod 2^N \oplus \lfloor \frac{j}{2^M} \rfloor) \cdot 2^M + (\lfloor \frac{i}{2^N} \rfloor \oplus j \bmod 2^M) + 1$ ，注意 i, j 从 0 开始。

14.7 高精度进制转换

2 — 62 进制相互转换。输入格式：“转换前进制 转换后进制 要转换的数据”。注释：进制排序为 0-9, A-Z, a-z。

```

1 struct numpy {
2     vector<int> mp; // 将字符转化为数字
3     vector<char> mp2; // 将数字转化为字符
4     numpy() : mp(123), mp2(62) { // 0-9A-Za-z
5         for (int i = 0; i < 10; i++) mp[i + 48] = i, mp2[i] = i + 48;
6         for (int i = 10; i < 36; i++) mp[i + 55] = i, mp2[i] = i + 55;
7         for (int i = 36; i < 62; i++) mp[i + 61] = i, mp2[i] = i + 61;
8     }
9
10    // 转换前进制 转换后进制 要转换的数据
11    string solve(int a, int b, const string &s) {
12        vector<int> nums, ans;
13        for (auto c : s) {
14            nums.push_back(mp[c]);
15        }
16        reverse(nums.begin(), nums.end());
17        while (nums.size()) { // 短除法, 将整个大数一直除 b, 取余数
18            int remainder = 0;
19            for (int i = nums.size() - 1; ~i; i--) {
20                nums[i] += remainder * a;
21                remainder = nums[i] % b;
22                nums[i] /= b;
23            }
24            ans.push_back(remainder); // 得到余数
25            while (nums.size() && nums.back() == 0) {
26                nums.pop_back(); // 去掉前导 0
27            }
28        }
29        reverse(ans.begin(), ans.end());
30
31        string sh;
32        for (int i : ans) sh += mp2[i];
33        return sh;
34    }
35 };

```

14.8 物品装箱

有 N 个物品, 第 i 个物品为 $a[i]$, 有无限个容量为 C 的空箱子。两种装箱方式, 输出需要多少个箱子才能装完所有物品。

14.8.1 从前往后装 (线段树解)

选择最前面的能放下物品的箱子放入物品。

```

1 const int N = 1e6 + 10;
2 int T, n, a[N], c, tr[N << 2];
3 void pushup(int u){
4     tr[u] = max(tr[u << 1], tr[u << 1 | 1]);
5 }
6 void build(int u, int l, int r){
7     if (l == r) tr[u] = c;
8     else {
9         int mid = l + r >> 1;
10        build(u << 1, l, mid);
11        build(u << 1 | 1, mid + 1, r);

```

```

12     pushup(u);
13 }
14 }
15 void update(int u, int l, int r, int p, int k){
16     if (l > p || r < p) return;
17     if (l == r) tr[u] -= k;
18     else {
19         int mid = l + r >> 1;
20         update(u << 1, l, mid, p, k);
21         update(u << 1 | 1, mid + 1, r, p, k);
22         pushup(u);
23     }
24 }
25 int query(int u, int l, int r, int k){
26     if (l == r){
27         if (tr[u] >= k) return 1;
28         return n + 1;
29     }
30     int mid = l + r >> 1;
31     if (tr[u << 1] >= k) return query(u << 1, l, mid, k);
32     else return query(u << 1 | 1, mid + 1, r, k);
33 }
34 int main() {
35     cin >> n >> c;
36     for (int i = 1; i <= n; i++) cin >> a[i];
37     build(1, 1, n);
38     for (int i = 1; i <= n; i++)
39         update(1, 1, n, query(1, 1, n, a[i]), a[i]);
40     cout << query(1, 1, n, c) - 1 << " ";
41 }

```

14.8.2 选择最优的箱子装 (multiset 解)

选择能放下物品且剩余容量最小的箱子放物品

```

1 void solve(){
2     cin >> n >> c;
3     for (int i = 1; i <= n; i++) cin >> a[i];
4     multiset<int> s;
5     for (int i = 1; i <= n; i++){
6         auto it = s.lower_bound(a[i]);
7         if (it == s.end()) s.insert(c - a[i]);
8         else {
9             int x = *it;
10             // multiset 可以存放重复数据，如果是删除某个值的话，会去掉多个箱子
11             // 导致答案错误，所以直接删除对应位置的元素
12             s.erase(it);
13             s.insert(x - a[i]);
14         }
15     }
16     cout << s.size() << "\n";
17 }

```

14.9 浮点数比较

比较下列浮点数的大小： x^{y^z} , x^{zy} , $(x^y)^z$, $(x^z)^y$, y^{xz} , y^{zx} , $(y^x)^z$, $(y^z)^x$, z^{xy} , z^{yx} , $(z^x)^y$ 和 $(z^y)^x$ 。

```

1 vector<pair<ld, int>> val = {
2     {log(x) * pow(y, z), 0}, {log(x) * pow(z, y), 1}, {log(x) * y * z, 2},
3     {log(x) * z * y, 3},    {log(y) * pow(x, z), 4}, {log(y) * pow(z, x), 5},
4     {log(y) * x * z, 6},    {log(y) * z * x, 7},    {log(z) * pow(x, y), 8},
5     {log(z) * pow(y, x), 9}, {log(z) * x * y, 10},   {log(z) * y * x, 11}};
6
7 sort(val.begin(), val.end(), [&](auto x, auto y) {
8     if (equal(x.first, y.first)) return x.second < y.second; // equal比较两个浮点数是
    否相等
9     return x.first > y.first;
10 });
11 cout << ans[val.front().second] << endl;

```

/END/

14.10 阿达马矩阵 (Hadamard matrix)

构造题用，其有一些性质：将 0 看作 -1 ；1 看作 $+1$ ，整个矩阵可以构成一个 2^k 维向量组，任意两个行、列向量的点积均为 0 [See](#)。例如，在 $k = 2$ 时行向量 $\vec{2}$ 和行向量 $\vec{3}$ 的点积为 $1 \cdot 1 + (-1) \cdot 1 + 1 \cdot (-1) + (-1) \cdot (-1) = 0$ 。

构造方式：
$$\begin{bmatrix} T & T \\ T & !T \end{bmatrix}$$

```
"1111",
"1010",
"11", "1100",
"10", "1001",
```

```
1  int n;
2  cin >> n;
3  int N = pow(2, n);
4  vector ans(N, vector<int>(N));
5  ans[0][0] = 1;
6  for (int t = 0; t < n; t++) {
7      int m = pow(2, t);
8      for (int i = 0; i < m; i++) {
9          for (int j = m; j < 2 * m; j++) {
10             ans[i][j] = ans[i][j - m];
11         }
12     }
13     for (int i = m; i < 2 * m; i++) {
14         for (int j = 0; j < m; j++) {
15             ans[i][j] = ans[i - m][j];
16         }
17     }
18     for (int i = m; i < 2 * m; i++) {
19         for (int j = m; j < 2 * m; j++) {
20             ans[i][j] = 1 - ans[i - m][j - m];
21         }
22     }
23 }
```

14.11 幻方

构造题用，其有一些性质（保证 N 为奇数）：1 到 N^2 每个数字恰好使用一次，且每行、每列及两条对角线上的数字之和都相同，且为奇数 [See](#)。

构造方式：将 1 写在第一行的中间，随后不断向右上角位置填下一个数字，直到填满。

```
"8 1 6",
"3 5 7",
"4 9 2",
```



```

1  int n;
2  cin >> n;
3  int x = 1, y = (n + 1) / 2;
4  vector<int> ans(n + 1, vector<int>(n + 1));
5  for (int i = 1; i <= n * n; i++) {
6      ans[x][y] = i;
7      if (!ans[(x - 2 + n) % n + 1][y % n + 1]){
8          x = (x - 2 + n) % n + 1;
9          y = y % n + 1;
10     } else {
11         x = x % n + 1;
12     }
13 }

```

14.12 最长严格/非严格递增子序列 (LIS)

14.12.1 一维

注意子序列是不连续的。使用二分搜索，以 $\mathcal{O}(N \log N)$ 复杂度通过，另也有 $\mathcal{O}(N^2)$ 的 dp 解法。

dis dis

```

1  vector<int> val; // 堆数
2  for (int i = 1, x; i <= n; i++) {
3      cin >> x;
4      int it = upper_bound(val.begin(), val.end(), x) - val.begin(); // low/up: 严格/
非严格递增
5      if (it >= val.size()) { // 新增一堆
6          val.push_back(x);
7      } else { // 更新对应位置元素
8          val[it] = x;
9      }
10 }
11 cout << val.size() << endl;

```

14.12.2 二维+输出方案

```

1  vector<array<int, 3>> in(n + 1);
2  for (int i = 1; i <= n; i++) {
3      cin >> in[i][0] >> in[i][1];
4      in[i][2] = i;
5  }
6  sort(in.begin() + 1, in.end(), [&](auto x, auto y) {
7      if (x[0] != y[0]) return x[0] < y[0];
8      return x[1] > y[1];
9  });
10
11 vector<int> val{0}, idx{0}, pre(n + 1);
12 for (int i = 1; i <= n; i++) {
13     auto [x, y, z] = in[i];
14     int it = lower_bound(val.begin(), val.end(), y) - val.begin(); // low/up: 严格/
非严格递增
15     if (it >= val.size()) { // 新增一堆
16         pre[z] = idx.back();
17         val.push_back(y);
18         idx.push_back(z);
19     } else { // 更新对应位置元素
20         pre[z] = idx[it - 1];
21         val[it] = y;
22         idx[it] = z;
23     }

```

```

24 }
25
26 vector<int> ans;
27 for (int i = idx.back(); i != 0; i = pre[i]) {
28     ans.push_back(i);
29 }
30 reverse(ans.begin(), ans.end());
31 cout << ans.size() << "\n";
32 for (auto it : ans) {
33     cout << it << " ";
34 }

```

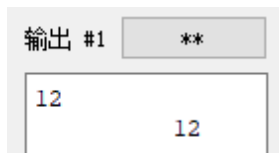
14.13 cout 输出流控制

设置字段宽度： `setw(x)`，该函数可以使得补全 x 位输出，默认用空格补全。

```

1 bool Solve() {
2     cout << 12 << endl;
3     cout << setw(12) << 12 << endl;
4     return 0;
5 }

```



输出 #1 **

```

12
12

```

设置填充字符： `setfill(x)`，该函数可以设定补全类型，注意这里的 x 只能为 `char` 类型。

```

1 bool Solve() {
2     cout << 12 << endl;
3     cout << setw(12) << setfill('*') << 12 << endl;
4     return 0;
5 }

```



输出 #1 **

```

12
*****12

```

14.14 读取一行数字，个数未知

```

1 string s;
2 getline(cin, s);
3 stringstream ss;
4 ss << s;
5 while (ss >> s) {
6     auto res = stoi(s);
7     cout << res * 100 << endl;
8 }

```

14.15 约瑟夫问题

n 个人编号 $0, 1, 2, \dots, n-1$ ，每次数到 k 出局，求最后剩下的人的编号。

$\mathcal{O}(N)$ 。

```
1 int jos(int n,int k){
2     int res=0;
3     repeat(i,1,n+1)res=(res+k)%i;
4     return res; // res+1, 如果编号从1开始
5 }
```

$\mathcal{O}(K \log N)$ ，适用于 K 较小的情况。

```
1 int jos(int n,int k){
2     if(n==1 || k==1)return n-1;
3     if(k>n)return (jos(n-1,k)+k)%n; // 线性算法
4     int res=jos(n-n/k,k)-n%k;
5     if(res<0)res+=n; // mod n
6     else res+=res/(k-1); // 还原位置
7     return res; // res+1, 如果编号从1开始
8 }
```

14.16 日期换算（基姆拉尔森公式）

已知年月日，求星期数。

```
1 int week(int y,int m,int d){
2     if(m<=2)m+=12,y--;
3     return (d+2*m+3*(m+1)/5+y+y/4-y/100+y/400)%7+1;
4 }
```

14.17 单调队列

查询区间 k 的最大最小值。

```
1 deque<int> D;
2 int n,k,x,a[MAX];
3 int main(){
4     IOS();
5     cin>>n>>k;
6     for(int i=1;i<=n;i++) cin>>a[i];
7     for(int i=1;i<=n;i++){
8         while(!D.empty() && a[D.back()]<=a[i]) D.pop_back();
9         D.emplace_back(i);
10        if(!D.empty()) if(i-D.front())>=k D.pop_front();
11        if(i>=k)cout<<a[D.front()]<<endl;
12    }
13    return 0;
14 }
```

14.18 高精度快速幂

求解 $n^k \bmod p$ ，其中 $0 \leq n, k \leq 10^{1000000}$ ， $1 \leq p \leq 10^9$ 。容易发现 n 可以直接取模，瓶颈在于 k [See](#)。

14.18.1 魔改十进制快速幂（暴力计算）

该算法复杂度 $\mathcal{O}(\text{len}(k))$ 。

```

1  int mypow10(int n, vector<int> k, int p) {
2      int r = 1;
3      for (int i = k.size() - 1; i >= 0; i--) {
4          for (int j = 1; j <= k[i]; j++) {
5              r = r * n % p;
6          }
7          int v = 1;
8          for (int j = 0; j <= 9; j++) {
9              v = v * n % p;
10             }
11             n = v;
12         }
13         return r;
14     }
15     signed main() {
16         string n_, k_;
17         int p;
18         cin >> n_ >> k_ >> p;
19
20         int n = 0; // 转化并计算 n % p
21         for (auto it : n_) {
22             n = n * 10 + it - '0';
23             n %= p;
24         }
25         vector<int> k; // 转化 k
26         for (auto it : k_) {
27             k.push_back(it - '0');
28         }
29         cout << mypow10(n, k, p) << endl; // 暴力快速幂
30     }

```

14.18.2 扩展欧拉定理（欧拉降幂公式）

$$n^k \equiv \begin{cases} n^{k \bmod \varphi(p)} & \gcd(n, p) = 1 \\ n^{k \bmod \varphi(p) + \varphi(p)} & \gcd(n, p) \neq 1 \wedge k \geq \varphi(p) \\ n^k & \gcd(n, p) \neq 1 \wedge k < \varphi(p) \end{cases}$$

最终我们可以将幂降到 $\varphi(p)$ 的级别，使得能够直接使用快速幂解题，复杂度瓶颈在求解欧拉函数 $\mathcal{O}(\sqrt{p})$ 。

```

1  int phi(int n) { //求解 phi(n)
2      int ans = n;
3      for (int i = 2; i <= n / i; i++) {
4          if (n % i == 0) {
5              ans = ans / i * (i - 1);
6              while (n % i == 0) {
7                  n /= i;
8              }
9          }
10     }
11     if (n > 1) { //特判 n 为质数的情况
12         ans = ans / n * (n - 1);
13     }
14     return ans;
15 }
16 signed main() {
17     string n_, k_;

```

```

18     int p;
19     cin >> n_ >> k_ >> p;
20
21     int n = 0; // 转化并计算 n % p
22     for (auto it : n_) {
23         n = n * 10 + it - '0';
24         n %= p;
25     }
26     int mul = phi(p), type = 0, k = 0; // 转化 k
27     for (auto it : k_) {
28         k = k * 10 + it - '0';
29         type |= (k >= mul);
30         k %= mul;
31     }
32     if (type) {
33         k += mul;
34     }
35     cout << mypow(n, k, p) << endl;
36 }

```

14.19 int128 输入输出流控制

int128 只在基于 Lumix 系统的环境下可用，需要 C++20。38位精度，除输入输出外与普通数据类型无差别。该封装支持负数读入，需要注意 `write` 函数结尾不输出多余空格与换行。

```

1  namespace my128 { // 读入优化封装，支持__int128
2      using i64 = __int128_t;
3      i64 abs(const i64 &x) {
4          return x > 0 ? x : -x;
5      }
6      auto &operator>>(istream &it, i64 &j) {
7          string val;
8          it >> val;
9          reverse(val.begin(), val.end());
10         i64 ans = 0;
11         bool f = 0;
12         char c = val.back();
13         val.pop_back();
14         for (; c < '0' || c > '9'; c = val.back(), val.pop_back()) {
15             if (c == '-') {
16                 f = 1;
17             }
18         }
19         for (; c >= '0' && c <= '9'; c = val.back(), val.pop_back()) {
20             ans = ans * 10 + c - '0';
21         }
22         j = f ? -ans : ans;
23         return it;
24     }
25     auto &operator<<(ostream &os, const i64 &j) {
26         string ans;
27         function<void(i64)> write = [&](i64 x) {
28             if (x < 0) ans += '-', x = -x;
29             if (x > 9) write(x / 10);
30             ans += x % 10 + '0';
31         };
32         write(j);
33         return os << ans;
34     }
35 } // namespace my128

```

14.20 对拍版子

- 文件控制

```

1 // BAD.cpp, 存放待寻找错误的代码
2 freopen("A.txt", "r", stdin);
3 freopen("BAD.out", "w", stdout);
4
5 // 1.cpp, 存放暴力或正确的代码
6 freopen("A.txt", "r", stdin);
7 freopen("1.out", "w", stdout);
8
9 // Ask.cpp
10 freopen("A.txt", "w", stdout);

```

- C++ 版 bat

```

1 int main() {
2     int T = 1E5;
3     while(T--) {
4         system("BAD.exe");
5         system("1.exe");
6         system("A.exe");
7         if (system("fc BAD.out 1.out")) {
8             puts("WA");
9             return 0;
10        }
11    }
12 }

```

14.21 随机数生成与样例构造

```

1
2 mt19937 rnd(chrono::steady_clock::now().time_since_epoch().count());
3 int r(int a, int b) {
4     return rnd() % (b - a + 1) + a;
5 }
6
7 void graph(int n, int root = -1, int m = -1) {
8     vector<pair<int, int>> t;
9     for (int i = 1; i < n; i++) { // 先建立一棵以0为根节点的树
10         t.emplace_back(i, r(0, i - 1));
11     }
12
13     vector<pair<int, int>> edge;
14     set<pair<int, int>> uni;
15     if (root == -1) root = r(0, n - 1); // 确定根节点
16     for (auto [x, y] : t) { // 偏移建树
17         x = (x + root) % n + 1;
18         y = (y + root) % n + 1;
19         edge.emplace_back(x, y);
20         uni.emplace(x, y);
21     }
22
23     if (m != -1) { // 如果是图，则在树的基础上继续加边
24         for (int i = n; i <= m; i++) {
25             while (true) {
26                 int x = r(1, n), y = r(1, n);
27                 if (x == y) continue; // 拒绝自环
28                 if (uni.count({x, y})) continue; // 拒绝重边

```

```

29         edge.emplace_back(x, y);
30         uni.emplace(x, y);
31     }
32 }
33 }
34
35 random_shuffle(edge.begin(), edge.end()); // 打乱节点
36 for (auto [x, y] : edge) {
37     cout << x << " " << y << endl;
38 }
39 }

```

14.22 手工哈希

```

1 struct myhash {
2     static uint64_t hash(uint64_t x) {
3         x += 0x9e3779b97f4a7c15;
4         x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
5         x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
6         return x ^ (x >> 31);
7     }
8     size_t operator()(uint64_t x) const {
9         static const uint64_t SEED =
10 chrono::steady_clock::now().time_since_epoch().count();
11         return hash(x + SEED);
12     }
13     size_t operator()(pair<uint64_t, uint64_t> x) const {
14         static const uint64_t SEED =
15 chrono::steady_clock::now().time_since_epoch().count();
16         return hash(x.first + SEED) ^ (hash(x.second + SEED) >> 1);
17     }
18 };
19 // unordered_map<int, int, myhash>

```

14.23 Python常用语法

14.23.1 读入与定义

- 读入多个变量并转换类型：`X, Y = map(int, input().split())`
- 读入列表：`X = eval(input())`
- 多维数组定义：`X = [[0 for j in range(0, 100)] for i in range(0, 200)]`

14.23.2 格式化输出

- 保留小数输出：`print("{:.12f}".format(X))` 指保留 12 位小数
- 对齐与宽度：`print("{:<12f}".format(X))` 指左对齐，保留 12 个宽度

14.23.3 排序

- 倒序排序：使用 `reverse` 实现倒序 `X.sort(reverse=True)`
- 自定义排序：下方代码实现了先按第一关键字降序、再按第二关键字升序排序。

```

1 X.sort(key=lambda x: x[1])
2 X.sort(key=lambda x: x[0], reverse=True)

```

14.23.4 文件IO

- 打开要读取的文件：`r = open('X.txt', 'r', encoding='utf-8')`
- 打开要写入的文件：`w = open('Y.txt', 'w', encoding='utf-8')`
- 按行写入：`w.write(XX)`

14.23.5 增加输出流长度、递归深度

```
1 import sys
2 sys.set_int_max_str_digits(200000)
3 sys.setrecursionlimit(100000)
```

14.23.6 自定义结构体

自定义结构体并且自定义排序

```
1 class node:
2     def __init__(self, A, B, C):
3         self.A = A
4         self.B = B
5         self.C = C
6
7 w = []
8 for i in range(1, 5):
9     a, b, c = input().split()
10    w.append(node(a, b, c))
11 w.sort(key=lambda x: x.C, reverse=True)
12 for i in w:
13     print(i.A, i.B, i.C)
14
```

14.23.7 数据结构

- 模拟于 C_{++}^{map} ，定义：`dic = dict()`
- 模拟栈与队列：使用常见的 `list` 即可完成，`list.insert(0, x)` 实现头部插入、`list.pop()` 实现尾部弹出、`list.pop(0)` 实现头部弹出

14.23.8 其他

- 获取ASCII码：`ord()` 函数
- 转换为ASCII字符：`chr()` 函数

14.24 OJ测试

对于一个未知属性的OJ，应当在正式赛前进行以下全部测试：

14.24.1 GNU C++ 版本测试

```
1 for (int i : {1, 2}) {} // GNU C++11 支持范围表达式
2
3 auto cc = [&](int x) { x++; }; // GNU C++11 支持 auto 与 lambda 表达式
4 cc(2);
5
6 tuple<string, int, int> V; // GNU C++11 引入
7 array<int, 3> C; // GNU C++11 引入
8
9 auto dfs = [&](auto self, int x) -> void { // GNU C++14 支持 auto 自递归
10     if (x > 10) return;
```



```

11     self(self, x + 1);
12 };
13 dfs(dfs, 1);
14
15 vector in(1, vector<int>(1)); // GNU C++17 支持 vector 模板类型缺失
16
17 map<int, int> dic;
18 for (auto [u, v] : dic) {} // GNU C++17 支持 auto 解绑
19 dic.contains(12); // GNU C++20 支持 contains 函数
20
21 constexpr double Pi = numbers::pi; // C++20 支持

```

14.24.2 编译器位数测试

```

1 using i64 = __int128; // 64 位 GNU C++11 支持

```

14.24.3 评测器环境测试

Windows 系统输出 -1；反之则为一个随机数。

```

1 #define int long long
2 map<int, int> dic;
3 int x = dic.size() - 1;
4 cout << x << endl;

```

14.24.4 运算速度测试

	本地-20(64)	CodeForces-20(64)	AtCoder-20(64)	牛客-17(64)	学院OJ	CodeForces-17(32)	马踏集
4E3量级-硬跑	2454	2886	874	4121	4807	2854	4986
4E3量级-手动加速	556	686	873	1716	1982	2246	2119

```

1 // #pragma GCC optimize("Ofast", "unroll-loops")
2 #include <bits/stdc++.h>
3 using namespace std;
4
5 signed main() {
6     int n = 4E3, cnt = 0;
7     bitset<30> ans;
8     for (int i = 1; i <= n; i++) {
9         for (int j = 1; j <= n; j += 2) {
10             for (int k = 1; k <= n; k += 4) {
11                 ans |= i | j | k;
12                 cnt++;
13             }
14         }
15     }
16     cout << cnt << "\n";
17 }

```

```

1 // #pragma GCC optimize("Ofast", "unroll-loops")
2
3 #include <bits/stdc++.h>
4 using namespace std;
5 mt19937 rnd(chrono::steady_clock::now().time_since_epoch().count());

```

```
6
7 signed main() {
8     size_t n = 340000000, seed = 0;
9     for (int i = 1; i <= n; i++) {
10         seed ^= rnd();
11     }
12
13     return 0;
14 }
```

14.25 编译器设置

```
1 g++ -O2 -std=c++20 -pipe
2 -Wall -Wextra -Wconversion /* 这部分是警告相关，可能用不到 */
3 -fstack-protector
4 -Wl,--stack=268435456
```

/END/



Edited by *Wida*

Version: 1.8.8 (2024.11.19)

www.github.com/hh2048
www.cnblogs.com/WIDA